

1_Projet6_Rondeau_Cecile_1_notebook_112025 copie

February 4, 2026

```
[1]: # Etape 1 - Importation des librairies et chargement des fichiers
```

```
[2]: import pandas as pd
import plotly.express as px
import warnings
warnings.filterwarnings("ignore", category=UserWarning, module="openpyxl")
```

```
[3]: # Chargement des fichiers

df_erp = pd.read_excel('Copie de erp.xlsx')
df_web = pd.read_excel('Copie de web.xlsx')
df_liaison = pd.read_excel('Copie de liaison.xlsx')
```

```
[4]: # Analyse du fichier ERP

#Afficher les dimensions du dataset ERP

print("Le tableau comporte {} observation(s) ou article(s)".format(df_erp.
↪shape[0]))
print("Le tableau comporte {} colonne(s)".format(df_erp.shape[1]))
```

Le tableau comporte 825 observation(s) ou article(s)

Le tableau comporte 6 colonne(s)

```
[5]: df_erp.head()
```

```
[5]:   product_id  onsale_web  price  stock_quantity  stock_status  purchase_price
0         3847           1   24.2             16      instock         12.88
1         3849           1   34.3             10      instock         17.54
2         3850           1   20.8              0  outofstock         10.64
3         4032           1   14.1             26      instock          6.92
4         4039           1   46.0              3  outofstock         23.77
```

```
[6]: # Compter combien de doublons dans colonne product id
df_erp['product_id'].duplicated().sum()
```

```
[6]: np.int64(0)
```

```
[7]: # Afficher les valeurs distincts stock_status
df_erp['stock_status'].unique()
```

```
[7]: array(['instock', 'outofstock'], dtype=object)
```

```
[8]: df_erp.groupby('stock_status')['stock_quantity'].describe()
```

```
[8]:
```

	count	mean	std	min	25%	50%	75%	max
stock_status								
instock	733.0	24.309686	21.793200	0.0	10.0	21.0	31.0	145.0
outofstock	92.0	-0.086957	1.095968	-10.0	0.0	0.0	0.0	3.0

```
[9]: # La colonne stock_status contient deux valeurs distinctes : instock et
      ↳ outofstock.
      # Elle est directement liée à la colonne stock_quantity, car le statut du stock
      ↳ est censé dépendre du niveau de quantité disponible.
      # En théorie, un produit instock doit avoir une quantité strictement positive,
      ↳ et un produit outofstock une quantité égale à zéro.
      # Cependant, l'analyse montre plusieurs incohérences dans les données :
      ↳ certaines lignes instock présentent une quantité nulle, tandis que certaines
      ↳ lignes outofstock possèdent une quantité positive ou même négative.
      # Ces anomalies traduisent des problèmes de qualité des données et devront être
      ↳ corrigées avant toute analyse métier approfondie.
```

```
[10]: # Création d'une colonne "stock_status_2"
      # La valeur de cette deuxième colonne sera fonction de la valeur dans la
      ↳ colonne "stock_quantity"
      # Si la valeur de la colonne "stock_quantity" est nulle, renseigner
      ↳ "outofstock" sinon mettre "instock"

df_erp["stock_status_2"] = df_erp["stock_quantity"].apply(lambda x:
      ↳ "outofstock" if x == 0 else "instock")
```

```
[11]: df_erp[["stock_quantity", "stock_status", "stock_status_2"]].head(10)
```

```
[11]:
```

	stock_quantity	stock_status	stock_status_2
0	16	instock	instock
1	10	instock	instock
2	0	outofstock	outofstock
3	26	instock	instock
4	3	outofstock	instock
5	12	instock	instock
6	12	instock	instock
7	15	instock	instock
8	0	outofstock	outofstock
9	5	instock	instock

```
[12]: #Vérifions que les 2 colonnes sont identiques:
#Les 2 colonnes sont strictement identiques si les valeurs de chaque ligne sont
↳strictement identiques 2 à 2
#La comparaison de 2 colonnes peut se réaliser simplement avec l'instruction
↳ci-dessous:
df_erp["stock_status"] == df_erp["stock_status_2"]
```

```
[12]: 0      True
1      True
2      True
3      True
4     False
...
820    True
821    True
822    True
823    True
824    True
Length: 825, dtype: bool
```

```
[13]: (df_erp["stock_status_2"] == df_erp["stock_status"]).sum()
```

```
[13]: np.int64(821)
```

```
[14]: # Afficher les 4 lignes qui posent problème
df_erp[df_erp["stock_status_2"] != df_erp["stock_status"]]
```

```
[14]:   product_id  onsale_web  price  stock_quantity  stock_status \
4          4039          1   46.0              3   outofstock
398         4885          1   18.7              0    instock
449         4973          0   10.0             -10   outofstock
573         5700          1   44.5             -1   outofstock

   purchase_price  stock_status_2
4           23.77    instock
398           9.66  outofstock
449           4.96    instock
573          22.30    instock
```

```
[15]: # Extraire les lignes incohérentes dans un nouveau DataFrame
df_erp_incoherent = df_erp[df_erp["stock_status_2"] != df_erp["stock_status"]].
↳copy()
```

```
[16]: # Créer un DataFrame "nettoyé" sans ces lignes
df_erp_clean = df_erp[df_erp["stock_status_2"] == df_erp["stock_status"]].copy()
```

```
[17]: # la colonne stock_status ne se met pas à jour automatiquement
# supprimer la colonne stock_statuts_2
# Création d'une colonne "stock_status_2" dans le nouveau df_erp_clean
df_erp_clean["stock_status_2"] = df_erp_clean["stock_quantity"].apply(lambda x:
    ↪ "outofstock" if x == 0 else "instock")

# Aperçu des premières lignes pour vérifier
df_erp_clean[["stock_quantity", "stock_status", "stock_status_2"]].head(10)
```

```
[17]:      stock_quantity  stock_status  stock_status_2
0                16      instock      instock
1                10      instock      instock
2                 0  outofstock  outofstock
3                26      instock      instock
5                12      instock      instock
6                12      instock      instock
7                15      instock      instock
8                 0  outofstock  outofstock
9                 5      instock      instock
10               2      instock      instock
```

```
[18]: # Afficher les 4 lignes qui posent problème
# Vérification qu'il n'y a PLUS d'incohérences dans df_erp_clean
df_erp_clean[df_erp_clean["stock_status_2"] != df_erp_clean["stock_status"]]
```

```
[18]: Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price, stock_status_2]
Index: []
```

```
[19]: # 2.1.1 - Analyse exploratoire de chaque variable du fichier erp.xlsx
```

```
[20]: # 2.1.1.1 - Analyse de la variable PRIX
```

```
[21]: # Vérification des prix: Y a t-il des prix non renseignés, négatifs ou nuls?
print("Nombre d'articles avec un prix non renseigné: {}".
    ↪ format(df_erp_clean["price"].isna().sum()))
```

Nombre d'articles avec un prix non renseigné: 0

```
[22]: # Afficher le prix minimum de la colonne "price"
# Afficher le prix maximum de la colonne "price"
# Afficher les prix inférieurs à 0 (qu'est-ce qu'il faut en faire ?)
resultat_price = df_erp_clean["price"].agg(['min', 'max', 'count'])
print(resultat_price)
```

```
min      -20.0
max      225.0
```

```
count      821.0
Name: price, dtype: float64
```

```
[23]: df_erp_clean[df_erp_clean["price"] < 0]
```

```
[23]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
151	4233	0	-20.0	0	outofstock	
469	5017	0	-8.0	0	outofstock	
739	6594	0	-9.1	19	instock	

	purchase_price	stock_status_2
151	10.33	outofstock
469	4.34	outofstock
739	4.61	instock

```
[24]: # Stocker les prix négatifs dans df_erp_incoherent
# Les mettre à part
df_price_negatif = df_erp_clean[df_erp_clean["price"] < 0].copy()

# Les ajouter dans le DataFrame des incohérences
df_erp_incoherent = pd.concat([df_erp_incoherent, df_price_negatif], axis=0)

# Les supprimer du DataFrame clean
df_erp_clean = df_erp_clean[df_erp_clean["price"] >= 0].copy()

# Vérification
print("Nombre de lignes avec prix négatif :", len(df_price_negatif))
```

Nombre de lignes avec prix négatif : 3

```
[25]: df_erp_incoherent.head(10)
```

```
[25]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
4	4039	1	46.0	3	outofstock	
398	4885	1	18.7	0	instock	
449	4973	0	10.0	-10	outofstock	
573	5700	1	44.5	-1	outofstock	
151	4233	0	-20.0	0	outofstock	
469	5017	0	-8.0	0	outofstock	
739	6594	0	-9.1	19	instock	

	purchase_price	stock_status_2
4	23.77	instock
398	9.66	outofstock
449	4.96	instock
573	22.30	instock
151	10.33	outofstock
469	4.34	outofstock

739 4.61 instock

```
[26]: df_erp_clean[df_erp_clean["price"] < 0]
```

```
[26]: Empty DataFrame
      Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
      purchase_price, stock_status_2]
      Index: []
```

```
[27]: resultat_stock_quantity = df_erp_clean["stock_quantity"].agg(['min', 'max'])
      print(resultat_stock_quantity)
```

```
min      0
max     145
Name: stock_quantity, dtype: int64
```

```
[28]: nombre_lignes = len(df_erp_clean)
      print(f"Nombre de lignes dans df_erp_clean : {nombre_lignes}")
```

```
Nombre de lignes dans df_erp_clean : 818
```

```
[29]: # 2.1.1.3 - Analyse de la variable ONSALE_WEB
```

```
[30]: # Vérification de la colonne onsale_web et des valeurs qu'elle contient. Que
      ↪ signifient-elles?
      df_erp_clean['onsale_web'].unique()
```

```
[30]: array([1, 0])
```

```
[31]: # Compter combien de fois apparaît 1 et 0
      df_erp_clean["onsale_web"].value_counts()
```

```
[31]: onsale_web
      1      713
      0     105
      Name: count, dtype: int64
```

```
[32]: # supprimer la colonne stock_statuts_2
      df_erp_clean.drop(columns=["stock_status_2"], inplace=True)
```

```
[33]: df_erp_clean.head()
```

```
[33]:   product_id  onsale_web  price  stock_quantity  stock_status  purchase_price
0         3847           1   24.2             16      instock         12.88
1         3849           1   34.3             10      instock         17.54
2         3850           1   20.8              0  outofstock         10.64
3         4032           1   14.1             26      instock          6.92
5         4040           1   34.3             12      instock         18.25
```

```
[34]: # 2.1.1.4 - Analyse de la variable prix d'achat
```

```
[35]: # Vérification de la colonne purchase_price :  
# Afficher le ou les prix non renseignés dans la colonne "purchase_price"  
print("Nombre d'articles avec un prix d'achat non renseigné: {}".  
      ↪format(df_erp_clean["purchase_price"].isna().sum()))
```

Nombre d'articles avec un prix d'achat non renseigné: 0

```
[36]: print("Prix d'achat minimum: ", df_erp_clean["purchase_price"].min())
```

Prix d'achat minimum: 2.74

```
[37]: print("Prix d'achat maximum: ", df_erp_clean["purchase_price"].max())
```

Prix d'achat maximum: 137.81

```
[38]: df_erp_clean[df_erp_clean["purchase_price"] < 0]
```

```
[38]: Empty DataFrame  
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,  
purchase_price]  
Index: []
```

```
[39]: df_erp_clean.to_excel("df_erp_clean.xlsx", index=False)
```

```
[40]: # Analyse du fichier ERP  
  
# Afficher les dimensions du dataset ERP  
  
print("Le tableau comporte {} observation(s) ou article(s)".format(df_erp_clean.  
      ↪shape[0]))  
print("Le tableau comporte {} colonne(s)".format(df_erp_clean.shape[1]))
```

Le tableau comporte 818 observation(s) ou article(s)

Le tableau comporte 6 colonne(s)

```
[41]: df_erp_incoherent.to_excel("df_erp_incoherent.xlsx", index=False)
```

```
[42]: # Analyse du fichier ERP  
  
# Afficher les dimensions du dataset ERP  
  
print("Le tableau comporte {} observation(s) ou article(s)".  
      ↪format(df_erp_incoherent.shape[0]))  
print("Le tableau comporte {} colonne(s)".format(df_erp_incoherent.shape[1]))
```

Le tableau comporte 7 observation(s) ou article(s)

Le tableau comporte 7 colonne(s)

```
[43]: # 2.2 - Analyse exploratoire du fichier web.xlsx
```

```
[44]: # Dimension du dataset et Nombre d'observations
```

```
nb_lignes, nb_colonnes = df_web.shape
print(f"le Datafram web comporte {nb_lignes} lignes, observations.")
print(f"le Datafram web comporte {nb_colonnes} colonnes.\n")
```

le Datafram web comporte 1513 lignes, observations.

le Datafram web comporte 29 colonnes.

```
[45]: # Le nombre de valeurs présentes dans chacune des colonnes WEB(si une cellule
      ↪ contient des cases vides le nb sera < à 1513
print("Nombre de valeurs non nulles dans chaque colonne :")
print(df_web.count())
```

Nombre de valeurs non nulles dans chaque colonne :

sku	1428
virtual	1513
downloadable	1513
rating_count	1513
average_rating	1430
total_sales	1430
tax_status	716
tax_class	0
post_author	1430
post_date	1430
post_date_gmt	1430
post_content	0
product_type	1429
post_title	1430
post_excerpt	716
post_status	1430
comment_status	1430
ping_status	1430
post_password	0
post_name	1430
post_modified	1430
post_modified_gmt	1430
post_content_filtered	0
post_parent	1430
guid	1430
menu_order	1430
post_type	1430
post_mime_type	714
comment_count	1430
dtype:	int64


```
[46]: # Identifiées les colonnes numériques ne comportant que des 0 (NaN autorisés)
colonnes_all_zeros_num = [
    c for c in df_web.columns
    if pd.api.types.is_numeric_dtype(df_web[c]) # condition la colonne doit
    ↪etre de type numeric
    and df_web[c].fillna(0).eq(0).all() # eq(0) transforme en true/false après
    ↪avoir remplacé les vides par 0, est-ce que TOUT est égal à 0 ?
]

print("Colonnes numériques uniquement à 0 :", colonnes_all_zeros_num)
```

Colonnes numériques uniquement à 0 : ['virtual', 'downloadable', 'rating_count', 'average_rating', 'tax_class', 'post_content', 'post_password', 'post_content_filtered', 'post_parent', 'menu_order', 'comment_count']

```
[47]: # Liste de colonnes à supprimer - identifiées ligne 46 + non pertinentes pour
    ↪effectuer analyse
colonnes_a_supprimer = [
    'virtual', 'downloadable', 'rating_count', 'average_rating',
    ↪'tax_class', 'tax_status', 'tax_class', 'post_author', 'post_date_gmt',
    ↪
    ↪'post_content', 'post_excerpt', 'post_status', 'comment_status', 'ping_status', 'post_password',
    ↪'post_parent',
    ↪'guid', 'menu_order', 'post_type', 'post_mime_type', 'comment_count'
]

# Suppression des colonnes
df_web = df_web.drop(columns=colonnes_a_supprimer)

print("Colonnes restantes :", df_web.shape[1])
```

Colonnes restantes : 5

```
[48]: # combien de lignes comporte colonne sku
print("Total lignes :", df_web["sku"].shape[0])
```

Total lignes : 1513

```
[49]: # Visualisation des valeurs de la colonne sku
print(df_web["sku"].head(20))
```

```
0    11862
1    16057
2    14692
3    16295
4    15328
5    15471
6    16515
7    16246
```

```

8      NaN
9      13572
10     16513
11     16585
12     16269
13     15526
14     12869
15     15575
16     11586
17     14338
18     15425
19     16560
Name: sku, dtype: object

```

```

[50]: # Type de données général de la colonne sku (vu par pandas)
print("Type de la colonne sku :", df_web["sku"].dtype)

```

Type de la colonne sku : object

```

[51]: print("\nTypes Python présents dans sku :")
print(df_web["sku"].apply(type).value_counts())

```

```

Types Python présents dans sku :
sku
<class 'int'>      1424
<class 'float'>    85
<class 'str'>      4
Name: count, dtype: int64

```

```

[52]: # Lister les valeurs non 'int'
non_num_sku_str = df_web["sku"].apply(type) == str
print("Nombre de SKU non numériques (str) :", non_num_sku_str.sum())

df_web[non_num_sku_str][["sku"]]

```

Nombre de SKU non numériques (str) : 4

```

[52]:          sku
272      13127-1
842  bon-cadeau-25-euros
1117      13127-1
1387  bon-cadeau-25-euros

```

```

[53]: # Vérification que les 85 type float sont Nan
sku_manquants = df_web["sku"].isna()
print("Nombre de SKU manquants (NaN) :", sku_manquants.sum())

```

Nombre de SKU manquants (NaN) : 85

```
[54]: # créer df pour comprendre doublons
df_sku_doublons = (
    df_web[df_web["sku"].duplicated(keep=False)]
    .assign(sku_str=df_web["sku"].astype(str))
    .sort_values("sku_str")
    .drop(columns="sku_str")
)
# nombre de lignes concernées par les doublons
print(f"Nombre de lignes avec doublons de SKU : {len(df_sku_doublons)}")
df_sku_doublons
```

Nombre de lignes avec doublons de SKU : 1513

```
[54]:      sku  total_sales  post_date  product_type \
668   10014          10.0 2019-04-04 15:45:23      Gin
1030   10014          10.0 2019-04-04 15:45:23      Gin
748   10459           4.0 2018-04-13 15:58:19      Vin
887   10459           4.0 2018-04-13 15:58:19      Vin
1317  10775           6.0 2018-04-17 21:28:52      Vin
...   ...          ...          ...          ...
639   NaN           NaN           NaT           NaN
199   NaN           NaN           NaT           NaN
1244   NaN           NaN           NaT           NaN
1429   NaN           NaN           NaT           NaN
717   NaN           NaN           NaT           NaN

      post_title
668      Darnley's London Dry Gin Original
1030     Darnley's London Dry Gin Original
748  Alphonse Mellot Sancerre Rouge Génération XIX ...
887  Alphonse Mellot Sancerre Rouge Génération XIX ...
1317 Albert Mann Pinot Gris Vendanges Tardives Alte...
...   ...
639      NaN
199      NaN
1244      NaN
1429      NaN
717      NaN

[1513 rows x 5 columns]
```

```
[55]: df_web.loc[[639, 199, 1244, 1429]]
```

```
[55]:      sku  total_sales  post_date  product_type  post_title
639   NaN           NaN           NaT           NaN           NaN
199   NaN           NaN           NaT           NaN           NaN
1244  NaN           NaN           NaT           NaN           NaN
1429  NaN           NaN           NaT           NaN           NaN
```

```
[56]: # Nombre de SKU en double

nb_doublons = df_web["sku"].duplicated().sum()
print(f"{nb_doublons} Skus apparaissent au moins une fois en doublon dans le jeu de données.")
```

798 Skus apparaissent au moins une fois en doublon dans le jeu de données.

```
[57]: # Identifier les lignes sans code article (sku manquant)
lignes_sans_sku = df_web["sku"].isna()

# Créer le DataFrame avec uniquement ces lignes
df_sans_sku = df_web[lignes_sans_sku].copy()

# Vérification
print("Nombre de lignes sans SKU :", df_sans_sku.shape[0])
```

Nombre de lignes sans SKU : 85

```
[58]: # Vérification si prix négatif dans total_sales

df_web[df_web["total_sales"] < 0]
```

```
[58]:      sku  total_sales      post_date product_type \
1084  NaN        -56.0 2018-08-08 11:23:43         Vin
1087  NaN        -17.0 2018-07-31 12:07:23         Vin

                                post_title
1084  Pierre Jean Villa Condrieu Jardin Suspendu 2018
1087      Pierre Jean Villa Côte Rôtie Fongéant 2017
```

```
[59]: # verification des lignes vides

nb_lignes_vides = df_web.isna().all(axis=1).sum()
print("Nombre de lignes entièrement vides :", nb_lignes_vides)
```

Nombre de lignes entièrement vides : 83

```
[60]: # Nombre de lignes avant
print("Avant :", df_web.shape[0])

# Suppression des lignes complètement vides
df_web = df_web.dropna(how="all")

# Nombre de lignes après
print("Après :", df_web.shape[0])
```

Avant : 1513

Après : 1430

```
[61]: df_sans_sku.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 85 entries, 8 to 1457
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sku              0 non-null     object
1   total_sales      2 non-null     float64
2   post_date        2 non-null     datetime64[ns]
3   product_type     2 non-null     object
4   post_title       2 non-null     object
dtypes: datetime64[ns](1), float64(1), object(3)
memory usage: 4.0+ KB
```

```
[62]: print("CRÉATION DES DATAFRAMES df_web_clean et df_web_incoherent")

# =====
# 1) DÉTECTION DES ANOMALIES
# =====

# SKU : conversion numérique (tout ce qui n'est pas convertible -> NaN)
sku_num = pd.to_numeric(df_web["sku"], errors="coerce")

# SKU non numériques : sku est renseigné MAIS non convertible en nombre
# ex: "bon-cadeau-25-euros", "13127-1"
sku_non_numeriques_mask = sku_num.isna() & df_web["sku"].notna()

# SKU manquants (vrais NaN)
sku_manquants_mask = df_web["sku"].isna()

print(f"SKU non numériques identifiés : {sku_non_numeriques_mask.sum()} lignes")
print(f"SKU manquants identifiés : {sku_manquants_mask.sum()} lignes")

# Ventes négatives
ventes_negatives_mask = df_web["total_sales"] < 0
print(f"Ventes négatives identifiées : {ventes_negatives_mask.sum()} lignes")

# =====
# 2) BASE CANDIDATS CLEAN (filtre cohérent)
# =====

# On garde uniquement :
# - SKU convertibles en num (donc pas non numériques, pas manquants)
# - total_sales >= 0
df_web_candidates = df_web.loc[
    ~sku_non_numeriques_mask & ~sku_manquants_mask & ~ventes_negatives_mask
```

```

].copy()

# =====
# 3) DF CLEAN = 1 LIGNE PAR SKU (première)
# =====

df_web_clean = df_web_candidates.drop_duplicates(subset="sku", keep="first").
↳copy()

# =====
# 4) DF INCOHERENT = tout le reste
# =====

# Doublons (par index) parmi les candidats
doublons_numeriques = df_web_candidates.loc[
    ~df_web_candidates.index.isin(df_web_clean.index)
].copy()

# Lignes SKU non numériques / manquants / ventes négatives (depuis df_web_
↳original)
sku_non_numeriques_df = df_web.loc[sku_non_numeriques_mask].copy()
sku_manquants_df = df_web.loc[sku_manquants_mask].copy()
ventes_negatives_df = df_web.loc[ventes_negatives_mask].copy()

# Assemblage
df_web_incoherent = pd.concat(
    [doublons_numeriques, sku_non_numeriques_df, sku_manquants_df,
↳ventes_negatives_df],
    ignore_index=True
).drop_duplicates()

# =====
# 5) RÉSULTATS + CHECKS
# =====

print("\nRÉSULTATS :")
print(f" • df_web (original) : {len(df_web)} lignes")
print(f" • df_web_clean (propre) : {len(df_web_clean)} lignes")
print(f" • df_web_incoherent (incohérent) : {len(df_web_incoherent)} lignes")
print(f"   - dont {len(sku_non_numeriques_df)} SKU non numériques")
print(f"   - dont {len(sku_manquants_df)} SKU manquants")
print(f"   - dont {len(doublons_numeriques)} doublons de SKU numériques")
print(f"   - dont {len(ventes_negatives_df)} lignes avec ventes négatives")

```

```

# Vérifications finales
print("\nVérification finale :")
print(f"Ventes négatives dans df_web_clean : {(df_web_clean['total_sales'] < 0).
↳sum()}")
print(f"Ventes négatives dans df_web_incoherent :
↳{(df_web_incoherent['total_sales'] < 0).sum()}")

# Optionnel (très utile) : vérifier la cohérence globale clean + incoherent =
↳original
# Attention : on compare des indices (pas le nombre de lignes après concat)
# Ici df_web_incoherent est reconstruit, donc on vérifie plutôt le filtrage de
↳base :
print("\nCohérence filtre candidates :")
print(f"Candidats clean + (non candidats) = original ? {len(df_web_candidates)
↳+ (sku_non_numeriques_mask | sku_manquants_mask | ventes_negatives_mask).
↳sum() <= len(df_web)}")

```

CRÉATION DES DATAFRAMES df_web_clean et df_web_incoherent

SKU non numériques identifiés : 4 lignes

SKU manquants identifiés : 2 lignes

Ventes négatives identifiées : 2 lignes

RÉSULTATS :

- df_web (original) : 1430 lignes
- df_web_clean (propre) : 712 lignes
- df_web_incoherent (incohérent) : 717 lignes
 - dont 4 SKU non numériques
 - dont 2 SKU manquants
 - dont 712 doublons de SKU numériques
 - dont 2 lignes avec ventes négatives

Vérification finale :

Ventes négatives dans df_web_clean : 0

Ventes négatives dans df_web_incoherent : 2

Cohérence filtre candidates :

Candidats clean + (non candidats) = original ? True

```
[63]: df_web_incoherent.to_excel("df_web_incoherent.xlsx", index=False)
```

```
[64]: df_web_clean.to_excel("df_web_clean.xlsx", index=False)
```

```
[65]: # Dimension du dataset
```

```

df_web_clean.shape
print("Le tableau comporte {} observation(s) ou article(s)".format(df_web_clean.
↳shape[0]))

```

```
print("Le tableau comporte {} colonne(s)".format(df_web_clean.shape[1]))
```

Le tableau comporte 712 observation(s) ou article(s)

Le tableau comporte 5 colonne(s)

```
[66]: df_web_incoherent.to_excel(" df_web_incoherent.xlsx", index=False)
```

```
[67]: # 2.3 - Analyse exploratoire du fichier liaison.xlsx
```

```
[68]: # Dimension du dataset
```

```
df_liaison.shape
print("Le tableau comporte {} observation(s) ou article(s)".format(df_liaison.
↪shape[0]))
print("Le tableau comporte {} colonne(s)".format(df_liaison.shape[1]))
```

Le tableau comporte 825 observation(s) ou article(s)

Le tableau comporte 2 colonne(s)

```
[69]: df_liaison.head()
```

```
[69]:   id_web  product_id
0   15298         3847
1   15296         3849
2   15300         3850
3   19814         4032
4   19815         4039
```

```
[70]: # La nature des données dans chacune des colonnes
df_liaison.dtypes
```

```
[70]: id_web      object
product_id    int64
dtype: object
```

```
[71]: print("\nTypes Python présents dans id_web :")
print(df_liaison["id_web"].apply(type).value_counts())
```

Types Python présents dans id_web :

```
id_web
<class 'int'>      731
<class 'float'>    91
<class 'str'>      3
Name: count, dtype: int64
```

```
[72]: # Lister les valeurs non 'int'
mask_str = df_liaison["id_web"].apply(type).eq(str)
```



```
print(f"Nombre de id_web non numériques (str) : {mask_str.sum()}")
df_liaison.loc[mask_str, ["id_web"]]
```

Nombre de id_web non numériques (str) : 3

```
[72]:          id_web
      443  bon-cadeau-25-euros
      822      13127-1
      823      14680-1
```

```
[73]: # vérification des lignes vides

lignes_vides = df_liaison[df_liaison.isnull().all(axis=1)]

print(f"Nombre de lignes totalement vides : {len(lignes_vides)}")
lignes_vides
```

Nombre de lignes totalement vides : 0

```
[73]: Empty DataFrame
      Columns: [id_web, product_id]
      Index: []
```

```
[74]: # Le nombre de valeurs présentes dans chacune des colonnes
print(df_liaison.count())
print()
```

```
id_web      734
product_id  825
dtype: int64
```

```
[75]: # Les valeurs de la colonne "product_id" sont-elles toutes uniques?
prodid_verification_unique = df_liaison['product_id'].is_unique
print(f"Les product_id sont-ils tous uniques ? {prodid_verification_unique}")
```

Les product_id sont-ils tous uniques ? True

```
[76]: # Avons-nous des articles sans correspondance?

# Regarde les cases vides dans 'id_web'
articles_sans_correspondance = df_liaison['id_web'].isna()

# Compte combien il y en a
nb_sans_correspondance = articles_sans_correspondance.sum()

print(f"Nombre d'articles SANS correspondance : {nb_sans_correspondance}")
print(f"Nombre d'articles AVEC correspondance : {len(df_liaison) -
↳ nb_sans_correspondance}")
```

Nombre d'articles SANS correspondance : 91
Nombre d'articles AVEC correspondance : 734

```
[77]: # Afficher articles sans correspondances

articles_sans_correspondance = df_liaison[df_liaison["id_web"].isna()]

print(f"Nombre total d'articles sans correspondance web : ␣
↪{len(articles_sans_correspondance)}")

articles_sans_correspondance.head(10)
```

Nombre total d'articles sans correspondance web : 91

```
[77]:      id_web  product_id
19      NaN         4055
49      NaN         4090
50      NaN         4092
119     NaN         4195
131     NaN         4209
151     NaN         4233
184     NaN         4278
185     NaN         4279
234     NaN         4565
238     NaN         4577
```

```
[78]: # 1) id_web : unicité (en ignorant les NaN)
id_web_non_null = df_liaison["id_web"].dropna()
print("id_web non nuls uniques ? :", id_web_non_null.is_unique)

# Si pas unique : combien de doublons ?
doublons_id_web = id_web_non_null.duplicated().sum()
print("Nombre de doublons id_web (hors NaN) :", doublons_id_web)
```

id_web non nuls uniques ? : True
Nombre de doublons id_web (hors NaN) : 0

```
[79]: # Masque les Id_web non numérique type str

mask_str = df_liaison["id_web"].apply(type).eq(str)

# Création du dataframe incohérent et clean
df_liaison_incoherent = df_liaison.loc[mask_str].copy()
df_liaison_clean = df_liaison.loc[~mask_str].copy()

print(f"Lignes incohérentes (id_web str) : {len(df_liaison_incoherent)}") # 3
print(f"Lignes clean restantes : {len(df_liaison_clean)}")
```

Lignes incohérentes (id_web str) : 3

Lignes clean restantes : 822

```
[80]: # Dimension du dataset

df_liaison_clean.shape
print("Le tableau comporte {} observation(s) ou article(s)".
      ↪format(df_liaison_clean.shape[0]))
print("Le tableau comporte {} colonne(s)".format(df_liaison_clean.shape[1]))
```

Le tableau comporte 822 observation(s) ou article(s)

Le tableau comporte 2 colonne(s)

```
[81]: # Etape 3 - Jonction des fichiers
```

```
[82]: # Rappel nombre ligne et colonnes de df_erp_clean
```

```
nb_lignes, nb_colonnes = df_erp_clean.shape
print("Lignes :", nb_lignes)
print("Colonnes :", nb_colonnes)
```

Lignes : 818

Colonnes : 6

```
[83]: # Etape 3.1 - Jonction du fichier df_erp et df_liaison
```

```
df_erp_liaison = df_erp_clean.
↪merge(df_liaison_clean,how="outer",on="product_id",indicator=True)
```

```
[84]: df_erp_liaison["_merge"].value_counts()
```

```
[84]: _merge
both          815
right_only     7
left_only      3
Name: count, dtype: int64
```

```
[85]: # Cpmprendre et visualiser les 7 right only
df_erp_liaison[df_erp_liaison["_merge"] == "right_only"]
```

```
[85]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
4	4039	NaN	NaN	NaN	NaN	
151	4233	NaN	NaN	NaN	NaN	
398	4885	NaN	NaN	NaN	NaN	
449	4973	NaN	NaN	NaN	NaN	
469	5017	NaN	NaN	NaN	NaN	
573	5700	NaN	NaN	NaN	NaN	
739	6594	NaN	NaN	NaN	NaN	

	purchase_price	id_web	_merge
--	----------------	--------	--------

4	NaN	19815	right_only
151	NaN	NaN	right_only
398	NaN	14981	right_only
449	NaN	NaN	right_only
469	NaN	NaN	right_only
573	NaN	14736	right_only
739	NaN	NaN	right_only

```
[86]: # Extraire les incohérence dans un fichier
df_liaison_incoherent = df_erp_liaison[df_erp_liaison["_merge"] ==
↳ "right_only"].copy()
```

```
[87]: # Cmpprendre et visualiser les 3 left only
df_erp_liaison[df_erp_liaison["_merge"] == "left_only"]
```

```
[87]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
443	4954	1.0	25.0	23.0	instock	
822	7247	1.0	54.8	6.0	instock	
823	7329	0.0	26.5	14.0	instock	

	purchase_price	id_web	_merge
443	13.30	NaN	left_only
822	27.18	NaN	left_only
823	13.42	NaN	left_only

```
[88]: # Vérification du fichier DF_liaison_incoherent
df_liaison_incoherent.shape
```

```
[88]: (7, 8)
```

```
[89]: # Décision de conservation de 3 produits left_only

# Je conserve les 3 produits left_only suivants :

# Product ID      Stock      Statut
# 4954            23 unités    instock
# 7247            6 unités     instock
# 7329            14 unités     instock
# Justification :
# Ces produits apparaissent en octobre avec du stock disponible. Ils auraient
↳ dû être mis à la vente. Leur exclusion priverait l'analyse de données
↳ importantes, car chaque journée de stock mobilisé représente un coût
↳ d'opportunité. Leur inclusion permettra d'évaluer correctement la rotation
↳ du stock disponible et d'optimiser la gestion des réceptions futures.
```

```
[90]: # Création du df erp liaison propre
```

```
df_erp_liaison_clean = df_erp_liaison[df_erp_liaison["_merge"].isin(["both",
↪ "left_only"])] .copy()
```

```
[91]: # Vérification
df_erp_liaison_clean["_merge"].value_counts()
```

```
[91]: _merge
both          815
left_only      3
right_only     0
Name: count, dtype: int64
```

```
[92]: # Vérification des colonnes de chaque df
print(df_erp_liaison_clean.columns)
print(df_web.columns)
```

```
Index(['product_id', 'onsale_web', 'price', 'stock_quantity', 'stock_status',
      'purchase_price', 'id_web', '_merge'],
      dtype='object')
Index(['sku', 'total_sales', 'post_date', 'product_type', 'post_title'],
      dtype='object')
```

```
[93]: df_erp_liaison_clean.to_excel("df_erp_liaison_clean.xlsx", index=False)
```

```
[94]: # Nettoyer l'ancienne colonne de contrôle "merge"
df_erp_liaison_clean = df_erp_liaison_clean.drop(columns="_merge",
↪ errors="ignore")
```

```
[95]: # Dimension du dataset df_erp_liaison_clean

df_liaison.shape
print("Le tableau comporte {} observation(s) ou article(s)".
↪ format(df_erp_liaison_clean.shape[0]))
print("Le tableau comporte {} colonne(s)".format(df_erp_liaison_clean.shape[1]))
```

```
Le tableau comporte 818 observation(s) ou article(s)
Le tableau comporte 7 colonne(s)
```

```
[96]: # Rappel nombre ligne et colonnes de df_web_clean

nb_lignes, nb_colonnes = df_web_clean.shape
print("Lignes :", nb_lignes)
print("Colonnes :", nb_colonnes)
```

```
Lignes : 712
Colonnes : 5
```

```
[97]: # fusionner df_erp_liaison et df_web_clean avec clé id_web et sku
```

```
df_final = df_erp_liaison_clean.  
↳merge(df_web_clean,how="outer",left_on="id_web",right_on="sku",indicator=True)
```

```
[98]: # Rappel nombre ligne et colonnes de df_final
```

```
nb_lignes, nb_colonnes = df_final.shape  
print("Lignes :", nb_lignes)  
print("Colonnes :", nb_colonnes)
```

```
Lignes : 821  
Colonnes : 13
```

```
[99]: df_final["_merge"].value_counts()
```

```
[99]: _merge  
both          709  
left_only     109  
right_only      3  
Name: count, dtype: int64
```

```
[100]: df_final.to_excel("df_final.xlsx", index=False)
```

```
[101]: # Visualiser les left_only
```

```
df_left_only = df_final[df_final["_merge"] == "left_only"]  
df_left_only
```

```
[101]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
81	4741.0	0.0	12.4	0.0	outofstock	
127	5957.0	0.0	39.0	0.0	outofstock	
139	4289.0	0.0	22.8	0.0	outofstock	
180	4869.0	0.0	17.2	0.0	outofstock	
185	5955.0	0.0	27.3	0.0	outofstock	
..	...	...	...	...	...	
816	7201.0	0.0	31.0	18.0	instock	
817	7203.0	0.0	45.0	30.0	instock	
818	7204.0	0.0	45.0	9.0	instock	
819	7247.0	1.0	54.8	6.0	instock	
820	7329.0	0.0	26.5	14.0	instock	

	purchase_price	id_web	sku	total_sales	post_date	product_type	\
81	6.66	12601	NaN	NaN	NaT	NaN	
127	20.75	13577	NaN	NaN	NaT	NaN	
139	11.90	13771	NaN	NaN	NaT	NaN	
180	9.33	14360	NaN	NaN	NaT	NaN	
185	13.68	14377	NaN	NaN	NaT	NaN	
..	...	...	...	...	...	...	
816	16.02	NaN	NaN	NaN	NaT	NaN	
817	23.48	NaN	NaN	NaN	NaT	NaN	

818	24.18	NaN	NaN	NaN	NaT	NaN
819	27.18	NaN	NaN	NaN	NaT	NaN
820	13.42	NaN	NaN	NaN	NaT	NaN

	post_title	_merge
81	NaN	left_only
127	NaN	left_only
139	NaN	left_only
180	NaN	left_only
185	NaN	left_only
..	...	...
816	NaN	left_only
817	NaN	left_only
818	NaN	left_only
819	NaN	left_only
820	NaN	left_only

[109 rows x 13 columns]

```
[102]: # visualiser les 3 right_only
df_right_only = df_final[df_final["_merge"] == "right_only"]
df_right_only
```

```
[102]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
228	NaN	NaN	NaN	NaN	NaN	
267	NaN	NaN	NaN	NaN	NaN	
726	NaN	NaN	NaN	NaN	NaN	

	purchase_price	id_web	sku	total_sales	post_date	\
228	NaN	NaN	14736	8.0	2019-01-31 11:58:26	
267	NaN	NaN	14981	10.0	2018-05-11 13:57:21	
726	NaN	NaN	19815	3.0	2018-02-12 09:04:37	

	product_type	post_title	\
228	Vin	Gilles Robin Crozes-Hermitage Rouge "1920"	2016
267	Vin	Saumaize-Michelin Mâcon Vergisson Sur La Roche...	
726	Vin	Pierre Jean Villa Côte Rôtie Carmina	2017

	_merge
228	right_only
267	right_only
726	right_only

```
[103]: # on extrait uniquement les 3 incoherences de right only
df_incoh_web = df_final[df_final["_merge"] == "right_only"].copy()
```

```
[104]: # Ajouter les 3 lignes au df liaison incohérent déjà existant
df_liaison_incoherent = pd.concat([df_liaison_incoherent,
↳ df_incoh_web], ignore_index=True)

[105]: # Création du df_final_complet
df_final_clean = df_final[df_final["_merge"].isin(["both", "left_only"])] .copy()

[106]: # Vérification
df_final_clean["_merge"].value_counts()

[106]: _merge
both          709
left_only     109
right_only      0
Name: count, dtype: int64

[107]: # Voir tous les 109 left_only
left_only_complets = df_final_clean[df_final_clean["_merge"] == "left_only"].
↳ copy()

print("=== TOUS LES 109 LEFT_ONLY ===")
print(f"Total left_only : {len(left_only_complets)}")

# Afficher les colonnes importantes
pd.set_option('display.max_rows', None)
print(left_only_complets[[
    "product_id", "id_web", "stock_quantity", "stock_status",
    "price", "purchase_price", "total_sales"
]].to_string())
pd.reset_option('display.max_rows')

# Statistiques
print("\n=== STATISTIQUES DES 109 LEFT_ONLY ===")
print(f"Avec id_web : {left_only_complets['id_web'].notna().sum()}")
print(f"Sans id_web (id_web NaN) : {left_only_complets['id_web'].isna().sum()}")
print(f"Avec stock > 0 : {(left_only_complets['stock_quantity'] > 0).sum()}")
print(f"Stock = 0 : {(left_only_complets['stock_quantity'] == 0).sum()}")

=== TOUS LES 109 LEFT_ONLY ===
Total left_only : 109
   product_id id_web  stock_quantity stock_status  price  purchase_price
total_sales
81         4741.0  12601              0.0  outofstock   12.4             6.66
NaN
127        5957.0  13577              0.0  outofstock   39.0            20.75
NaN
139        4289.0  13771              0.0  outofstock   22.8            11.90
NaN
```


180	4869.0	14360	0.0	outofstock	17.2	9.33
NaN						
185	5955.0	14377	0.0	outofstock	27.3	13.68
NaN						
186	5953.0	14379	0.0	outofstock	47.5	23.81
NaN						
212	5505.0	14648	0.0	outofstock	10.1	5.22
NaN						
218	5800.0	14689	0.0	outofstock	32.3	16.02
NaN						
224	5559.0	14715	3.0	instock	27.9	13.98
NaN						
227	5570.0	14730	0.0	outofstock	22.5	11.16
NaN						
236	4584.0	14785	0.0	outofstock	32.3	17.36
NaN						
280	4568.0	15065	0.0	outofstock	21.5	11.22
NaN						
303	4864.0	15154	0.0	outofstock	8.3	9.99
NaN						
332	5018.0	15272	0.0	outofstock	15.4	7.72
NaN						
412	6100.0	15529	0.0	outofstock	12.9	6.47
NaN						
428	4922.0	15586	0.0	outofstock	21.5	10.55
NaN						
430	4921.0	15608	0.0	outofstock	13.8	7.13
NaN						
431	5954.0	15609	0.0	outofstock	18.8	9.32
NaN						
438	5021.0	15630	0.0	outofstock	17.1	8.92
NaN						
731	4055.0	NaN	0.0	outofstock	86.1	37.88
NaN						
732	4090.0	NaN	0.0	outofstock	73.0	33.79
NaN						
733	4092.0	NaN	0.0	outofstock	47.0	25.25
NaN						
734	4195.0	NaN	0.0	outofstock	14.1	7.36
NaN						
735	4209.0	NaN	0.0	outofstock	73.5	33.01
NaN						
736	4278.0	NaN	0.0	outofstock	21.5	10.78
NaN						
737	4279.0	NaN	0.0	outofstock	10.8	5.64
NaN						
738	4565.0	NaN	3.0	instock	30.5	15.92
NaN						

739	4577.0	NaN	1.0	instock	49.0	24.05
NaN						
740	4578.0	NaN	3.0	instock	40.0	20.05
NaN						
741	4594.0	NaN	0.0	outofstock	144.0	87.36
NaN						
742	4599.0	NaN	0.0	outofstock	36.9	18.30
NaN						
743	4659.0	NaN	0.0	outofstock	23.6	12.56
NaN						
744	4692.0	NaN	48.0	instock	12.0	6.39
NaN						
745	4693.0	NaN	0.0	outofstock	18.5	9.75
NaN						
746	4697.0	NaN	1.0	instock	34.5	18.72
NaN						
747	4698.0	NaN	0.0	outofstock	20.5	10.17
NaN						
748	4702.0	NaN	0.0	outofstock	23.8	11.93
NaN						
749	4721.0	NaN	0.0	outofstock	20.2	10.54
NaN						
750	4738.0	NaN	3.0	instock	13.5	6.77
NaN						
751	4744.0	NaN	5.0	instock	13.8	6.84
NaN						
752	4798.0	NaN	0.0	outofstock	12.7	6.30
NaN						
753	4874.0	NaN	0.0	outofstock	14.6	7.62
NaN						
754	4911.0	NaN	0.0	outofstock	23.0	12.00
NaN						
755	4954.0	NaN	23.0	instock	25.0	13.30
NaN						
756	5020.0	NaN	0.0	outofstock	10.0	5.27
NaN						
757	5070.0	NaN	0.0	outofstock	84.7	47.43
NaN						
758	5075.0	NaN	0.0	outofstock	43.3	21.70
NaN						
759	5560.0	NaN	62.0	instock	47.0	25.50
NaN						
760	5569.0	NaN	1.0	instock	19.9	9.77
NaN						
761	5805.0	NaN	0.0	outofstock	29.2	15.09
NaN						
762	5808.0	NaN	1.0	instock	34.2	18.55
NaN						

763	5952.0	NaN	0.0	outofstock	42.5	22.84
NaN						
764	6125.0	NaN	0.0	outofstock	14.2	7.26
NaN						
765	6324.0	NaN	18.0	instock	92.0	99.00
NaN						
766	6327.0	NaN	0.0	outofstock	28.5	14.43
NaN						
767	6821.0	NaN	21.0	instock	28.0	14.61
NaN						
768	6824.0	NaN	8.0	instock	28.0	14.32
NaN						
769	6825.0	NaN	5.0	instock	24.0	12.40
NaN						
770	6826.0	NaN	11.0	instock	18.0	8.93
NaN						
771	6864.0	NaN	32.0	instock	40.0	21.08
NaN						
772	6866.0	NaN	42.0	instock	40.0	20.67
NaN						
773	6869.0	NaN	4.0	instock	40.0	19.84
NaN						
774	6875.0	NaN	17.0	instock	40.0	20.46
NaN						
775	6898.0	NaN	51.0	instock	30.0	14.88
NaN						
776	6899.0	NaN	3.0	instock	26.0	13.70
NaN						
777	6900.0	NaN	0.0	outofstock	30.0	16.28
NaN						
778	6901.0	NaN	0.0	outofstock	20.0	10.64
NaN						
779	6902.0	NaN	6.0	instock	31.0	16.50
NaN						
780	6903.0	NaN	54.0	instock	27.0	14.23
NaN						
781	6904.0	NaN	22.0	instock	31.0	15.70
NaN						
782	6905.0	NaN	3.0	instock	21.0	10.63
NaN						
783	6906.0	NaN	12.0	instock	31.0	16.82
NaN						
784	6907.0	NaN	8.0	instock	27.0	13.67
NaN						
785	6908.0	NaN	14.0	instock	31.0	15.70
NaN						
786	6909.0	NaN	7.0	instock	21.0	11.28
NaN						

787	7008.0	NaN	5.0	instock	40.0	20.67
NaN						
788	7009.0	NaN	10.0	instock	40.0	19.63
NaN						
789	7010.0	NaN	37.0	instock	47.0	23.55
NaN						
790	7015.0	NaN	12.0	instock	45.0	23.48
NaN						
791	7081.0	NaN	17.0	instock	45.0	23.95
NaN						
792	7084.0	NaN	7.0	instock	45.0	24.18
NaN						
793	7085.0	NaN	8.0	instock	30.0	14.73
NaN						
794	7086.0	NaN	2.0	instock	26.0	13.43
NaN						
795	7087.0	NaN	11.0	instock	30.0	15.66
NaN						
796	7088.0	NaN	53.0	instock	20.0	10.44
NaN						
797	7131.0	NaN	19.0	instock	45.0	24.18
NaN						
798	7132.0	NaN	22.0	instock	45.0	24.18
NaN						
799	7133.0	NaN	26.0	instock	45.0	23.95
NaN						
800	7136.0	NaN	7.0	instock	45.0	22.55
NaN						
801	7137.0	NaN	3.0	instock	45.0	23.25
NaN						
802	7159.0	NaN	1.0	instock	31.0	16.18
NaN						
803	7161.0	NaN	5.0	instock	21.0	10.85
NaN						
804	7162.0	NaN	1.0	instock	27.0	13.67
NaN						
805	7163.0	NaN	42.0	instock	31.0	16.66
NaN						
806	7164.0	NaN	51.0	instock	31.0	16.02
NaN						
807	7168.0	NaN	29.0	instock	45.0	23.25
NaN						
808	7169.0	NaN	3.0	instock	45.0	24.41
NaN						
809	7170.0	NaN	43.0	instock	45.0	23.72
NaN						
810	7192.0	NaN	28.0	instock	31.0	16.50
NaN						

811	7193.0	NaN	14.0	instock	27.0	13.67
NaN						
812	7194.0	NaN	5.0	instock	31.0	16.02
NaN						
813	7195.0	NaN	1.0	instock	21.0	10.31
NaN						
814	7196.0	NaN	55.0	instock	31.0	31.20
NaN						
815	7200.0	NaN	6.0	instock	31.0	15.54
NaN						
816	7201.0	NaN	18.0	instock	31.0	16.02
NaN						
817	7203.0	NaN	30.0	instock	45.0	23.48
NaN						
818	7204.0	NaN	9.0	instock	45.0	24.18
NaN						
819	7247.0	NaN	6.0	instock	54.8	27.18
NaN						
820	7329.0	NaN	14.0	instock	26.5	13.42
NaN						

=== STATISTIQUES DES 109 LEFT_ONLY ===

Avec id_web : 19

Sans id_web (id_web NaN) : 90

Avec stock > 0 : 65

Stock = 0 : 44

```
[108]: # Identifier les lignes left_only avec des données MANQUANTES (NaN) dans les
        ↪ colonnes principales
left_only_vides = df_final_clean[df_final_clean["_merge"] == "left_only"].copy()

print(f"Total left_only : {len(left_only_vides)}")
print("Colonnes affichées : product_id, id_web, stock_quantity, stock_status,
        ↪ price, total_sales")
```

Total left_only : 109

Colonnes affichées : product_id, id_web, stock_quantity, stock_status, price, total_sales

```
[109]: # Identifier les left_only avec stock à 0

left_only_stock_0 = df_final[(df_final["_merge"] == "left_only")
        ↪ &(df_final["stock_quantity"] == 0)].copy()

left_only_stock_0[ ["product_id", "id_web", "stock_quantity", "stock_status",
        ↪ "price"]]
```

```
[109]:
```

	product_id	id_web	stock_quantity	stock_status	price
81	4741.0	12601	0.0	outofstock	12.4
127	5957.0	13577	0.0	outofstock	39.0
139	4289.0	13771	0.0	outofstock	22.8
180	4869.0	14360	0.0	outofstock	17.2
185	5955.0	14377	0.0	outofstock	27.3
186	5953.0	14379	0.0	outofstock	47.5
212	5505.0	14648	0.0	outofstock	10.1
218	5800.0	14689	0.0	outofstock	32.3
227	5570.0	14730	0.0	outofstock	22.5
236	4584.0	14785	0.0	outofstock	32.3
280	4568.0	15065	0.0	outofstock	21.5
303	4864.0	15154	0.0	outofstock	8.3
332	5018.0	15272	0.0	outofstock	15.4
412	6100.0	15529	0.0	outofstock	12.9
428	4922.0	15586	0.0	outofstock	21.5
430	4921.0	15608	0.0	outofstock	13.8
431	5954.0	15609	0.0	outofstock	18.8
438	5021.0	15630	0.0	outofstock	17.1
731	4055.0	NaN	0.0	outofstock	86.1
732	4090.0	NaN	0.0	outofstock	73.0
733	4092.0	NaN	0.0	outofstock	47.0
734	4195.0	NaN	0.0	outofstock	14.1
735	4209.0	NaN	0.0	outofstock	73.5
736	4278.0	NaN	0.0	outofstock	21.5
737	4279.0	NaN	0.0	outofstock	10.8
741	4594.0	NaN	0.0	outofstock	144.0
742	4599.0	NaN	0.0	outofstock	36.9
743	4659.0	NaN	0.0	outofstock	23.6
745	4693.0	NaN	0.0	outofstock	18.5
747	4698.0	NaN	0.0	outofstock	20.5
748	4702.0	NaN	0.0	outofstock	23.8
749	4721.0	NaN	0.0	outofstock	20.2
752	4798.0	NaN	0.0	outofstock	12.7
753	4874.0	NaN	0.0	outofstock	14.6
754	4911.0	NaN	0.0	outofstock	23.0
756	5020.0	NaN	0.0	outofstock	10.0
757	5070.0	NaN	0.0	outofstock	84.7
758	5075.0	NaN	0.0	outofstock	43.3
761	5805.0	NaN	0.0	outofstock	29.2
763	5952.0	NaN	0.0	outofstock	42.5
764	6125.0	NaN	0.0	outofstock	14.2
766	6327.0	NaN	0.0	outofstock	28.5
777	6900.0	NaN	0.0	outofstock	30.0
778	6901.0	NaN	0.0	outofstock	20.0

```
[110]: print("Nombre de lignes left_only avec stock à 0 :", left_only_stock_0.shape[0])
```

Nombre de lignes left_only avec stock à 0 : 44

```
[111]: # Sans informations complémentaires de la direction, j'ai extrait ces produits
      ↪ de l'analyse principale.

      # . Leur présence aurait perturbé l'analyse de la rotation des stocks et des
      ↪ marges, car ils ne présentent aucun mouvement de stock (0) ni quantité
      ↪ vendue en octobre.

      # Ils correspondent probablement à :

      # . - des références en cours d'arrêt,
      # . - des nouveautés en attente de lancement,
      # . ou des produits saisonniers pour les fêtes à venir.

      # par contre je conserve pour la même raison qu'évoquer les avec les stock > 0:
      ↪ 65 ligne 89 Ces produits apparaissent en octobre avec du stock disponible.
      ↪ Ils auraient dû être mis à la vente. Leur exclusion priverait l'analyse de
      ↪ données importantes, car chaque journée de stock mobilisé représente un coût
      ↪ d'opportunité. Leur inclusion permettra d'évaluer correctement la rotation
      ↪ du stock disponible et d'optimiser la gestion des réceptions futures.
```

```
[112]: # Supprimer les produits left_only avec stock = 0 du df_final_clean
df_final_clean = df_final_clean[
    ~((df_final_clean["_merge"] == "left_only") &
      ↪ (df_final_clean["stock_quantity"] == 0))
].copy()

# Vérification
print("Distribution après suppression :")
print(df_final_clean["_merge"].value_counts())
print(f"\nNombre total de lignes : {len(df_final_clean)}")

# Compter les left_only restants (avec stock > 0)
left_only_restant = df_final_clean[df_final_clean["_merge"] == "left_only"]
print(f"\nLeft_only avec stock > 0 : {len(left_only_restant)} lignes")
if len(left_only_restant) > 0:
    print("Exemple :")
    print(left_only_restant[["product_id", "id_web", "stock_quantity",
      ↪ "stock_status"]].head())
```

Distribution après suppression :

```
_merge
both          709
left_only      65
right_only     0
Name: count, dtype: int64
```

Nombre total de lignes : 774

Left_only avec stock > 0 : 65 lignes

Exemple :

	product_id	id_web	stock_quantity	stock_status
224	5559.0	14715	3.0	instock
738	4565.0	NaN	3.0	instock
739	4577.0	NaN	1.0	instock
740	4578.0	NaN	3.0	instock
744	4692.0	NaN	48.0	instock

```
[113]: # Analyser les 65 left_only restants
print("=== ANALYSE DES 65 LEFT_ONLY RESTANTS ===")
print("(Produits ERP avec stock > 0 mais sans correspondance web complète)")

# Statistiques détaillées
print("\n STATISTIQUES :")
print(f"1. Avec id_web : {left_only_restant['id_web'].notna().sum()} produits")
print(f"2. Sans id_web : {left_only_restant['id_web'].isna().sum()} produits")

# Détail par catégorie
avec_idweb = left_only_restant[left_only_restant['id_web'].notna()]
sans_idweb = left_only_restant[left_only_restant['id_web'].isna()]

print(f"\n AVEC id_web ({len(avec_idweb)} produits) :")
print(" Ces produits ont une liaison mais pas de données de vente web")
print(f" Stock total : {avec_idweb['stock_quantity'].sum():.0f} unités")
print(f" Valeur stock : {(avec_idweb['stock_quantity'] * avec_idweb['price']).
    ↳sum():.0f} €")
print(" Exemples :")
print(avec_idweb[['product_id', 'id_web', 'stock_quantity', 'price']].head())

print(f"\n SANS id_web ({len(sans_idweb)} produits) :")
print(" Ces produits ont du stock mais ne sont PAS sur le site web")
print(f" Stock total : {sans_idweb['stock_quantity'].sum():.0f} unités")
print(f" Valeur stock : {(sans_idweb['stock_quantity'] * sans_idweb['price']).
    ↳sum():.0f} €")
print(" Exemples :")
print(sans_idweb[['product_id', 'stock_quantity', 'price', 'stock_status']].
    ↳head())

# Détail complet des 65
print("\n" + "="*50)
print(" LISTE COMPLÈTE DES 65 LEFT_ONLY :")
print("="*50)

pd.set_option('display.max_rows', None)
```



```

print(left_only_restant[[
    'product_id', 'id_web', 'stock_quantity', 'stock_status',
    'price', 'purchase_price'
]].sort_values('stock_quantity', ascending=False).to_string())
pd.reset_option('display.max_rows')

# Recommandation
print(f"\n RECOMMANDATION :")
print(f"Vous avez {len(sans_idweb)} produits avec du stock mais ABSENTS du site_
↪web.")
print(f"Valeur du stock dormant : {(sans_idweb['stock_quantity'] *
↪sans_idweb['price']).sum():.0f} €")
print(f"→ Ces produits pourraient être référencés pour générer du CA_
↪supplémentaire.")

```

=== ANALYSE DES 65 LEFT_ONLY RESTANTS ===
(Produits ERP avec stock > 0 mais sans correspondance web complète)

STATISTIQUES :

1. Avec id_web : 1 produits
2. Sans id_web : 64 produits

AVEC id_web (1 produits) :

Ces produits ont une liaison mais pas de données de vente web

Stock total : 3 unités

Valeur stock : 84 €

Exemples :

	product_id	id_web	stock_quantity	price
224	5559.0	14715	3.0	27.9

SANS id_web (64 produits) :

Ces produits ont du stock mais ne sont PAS sur le site web

Stock total : 1089 unités

Valeur stock : 38128 €

Exemples :

	product_id	stock_quantity	price	stock_status
738	4565.0	3.0	30.5	instock
739	4577.0	1.0	49.0	instock
740	4578.0	3.0	40.0	instock
744	4692.0	48.0	12.0	instock
746	4697.0	1.0	34.5	instock

=====

LISTE COMPLÈTE DES 65 LEFT_ONLY :

=====

	product_id	id_web	stock_quantity	stock_status	price	purchase_price
759	5560.0	NaN	62.0	instock	47.0	25.50

814	7196.0	NaN	55.0	instock	31.0	31.20
780	6903.0	NaN	54.0	instock	27.0	14.23
796	7088.0	NaN	53.0	instock	20.0	10.44
806	7164.0	NaN	51.0	instock	31.0	16.02
775	6898.0	NaN	51.0	instock	30.0	14.88
744	4692.0	NaN	48.0	instock	12.0	6.39
809	7170.0	NaN	43.0	instock	45.0	23.72
805	7163.0	NaN	42.0	instock	31.0	16.66
772	6866.0	NaN	42.0	instock	40.0	20.67
789	7010.0	NaN	37.0	instock	47.0	23.55
771	6864.0	NaN	32.0	instock	40.0	21.08
817	7203.0	NaN	30.0	instock	45.0	23.48
807	7168.0	NaN	29.0	instock	45.0	23.25
810	7192.0	NaN	28.0	instock	31.0	16.50
799	7133.0	NaN	26.0	instock	45.0	23.95
755	4954.0	NaN	23.0	instock	25.0	13.30
781	6904.0	NaN	22.0	instock	31.0	15.70
798	7132.0	NaN	22.0	instock	45.0	24.18
767	6821.0	NaN	21.0	instock	28.0	14.61
797	7131.0	NaN	19.0	instock	45.0	24.18
765	6324.0	NaN	18.0	instock	92.0	99.00
816	7201.0	NaN	18.0	instock	31.0	16.02
791	7081.0	NaN	17.0	instock	45.0	23.95
774	6875.0	NaN	17.0	instock	40.0	20.46
785	6908.0	NaN	14.0	instock	31.0	15.70
811	7193.0	NaN	14.0	instock	27.0	13.67
820	7329.0	NaN	14.0	instock	26.5	13.42
790	7015.0	NaN	12.0	instock	45.0	23.48
783	6906.0	NaN	12.0	instock	31.0	16.82
795	7087.0	NaN	11.0	instock	30.0	15.66
770	6826.0	NaN	11.0	instock	18.0	8.93
788	7009.0	NaN	10.0	instock	40.0	19.63
818	7204.0	NaN	9.0	instock	45.0	24.18
793	7085.0	NaN	8.0	instock	30.0	14.73
784	6907.0	NaN	8.0	instock	27.0	13.67
768	6824.0	NaN	8.0	instock	28.0	14.32
786	6909.0	NaN	7.0	instock	21.0	11.28
792	7084.0	NaN	7.0	instock	45.0	24.18
800	7136.0	NaN	7.0	instock	45.0	22.55
819	7247.0	NaN	6.0	instock	54.8	27.18
779	6902.0	NaN	6.0	instock	31.0	16.50
815	7200.0	NaN	6.0	instock	31.0	15.54
769	6825.0	NaN	5.0	instock	24.0	12.40
803	7161.0	NaN	5.0	instock	21.0	10.85
787	7008.0	NaN	5.0	instock	40.0	20.67
751	4744.0	NaN	5.0	instock	13.8	6.84
812	7194.0	NaN	5.0	instock	31.0	16.02
773	6869.0	NaN	4.0	instock	40.0	19.84

750	4738.0	NaN	3.0	instock	13.5	6.77
808	7169.0	NaN	3.0	instock	45.0	24.41
740	4578.0	NaN	3.0	instock	40.0	20.05
738	4565.0	NaN	3.0	instock	30.5	15.92
801	7137.0	NaN	3.0	instock	45.0	23.25
776	6899.0	NaN	3.0	instock	26.0	13.70
782	6905.0	NaN	3.0	instock	21.0	10.63
224	5559.0	14715	3.0	instock	27.9	13.98
794	7086.0	NaN	2.0	instock	26.0	13.43
760	5569.0	NaN	1.0	instock	19.9	9.77
762	5808.0	NaN	1.0	instock	34.2	18.55
804	7162.0	NaN	1.0	instock	27.0	13.67
802	7159.0	NaN	1.0	instock	31.0	16.18
813	7195.0	NaN	1.0	instock	21.0	10.31
746	4697.0	NaN	1.0	instock	34.5	18.72
739	4577.0	NaN	1.0	instock	49.0	24.05

RECOMMANDATION :

Vous avez 64 produits avec du stock mais ABSENTS du site web.

Valeur du stock dormant : 38128 €

→ Ces produits pourraient être référencés pour générer du CA supplémentaire.

```
[114]: df_liaison_incoherent[["product_id", "id_web", "stock_quantity", "stock_status", "_merge"]]
```

```
[114]:
```

	product_id	id_web	stock_quantity	stock_status	_merge
0	4039.0	19815	NaN	NaN	right_only
1	4233.0	NaN	NaN	NaN	right_only
2	4885.0	14981	NaN	NaN	right_only
3	4973.0	NaN	NaN	NaN	right_only
4	5017.0	NaN	NaN	NaN	right_only
5	5700.0	14736	NaN	NaN	right_only
6	6594.0	NaN	NaN	NaN	right_only
7	NaN	NaN	NaN	NaN	right_only
8	NaN	NaN	NaN	NaN	right_only
9	NaN	NaN	NaN	NaN	right_only

```
[115]: # Vérification
df_final_clean["_merge"].value_counts()
```

```
[115]: _merge
both      709
left_only   65
right_only   0
Name: count, dtype: int64
```

```
[116]: print("df_final_clean :", df_final_clean.shape)
```

```
df_final_clean : (774, 13)
```

```
[117]: df_final_clean.columns
```

```
[117]: Index(['product_id', 'onsale_web', 'price', 'stock_quantity', 'stock_status',  
        'purchase_price', 'id_web', 'sku', 'total_sales', 'post_date',  
        'product_type', 'post_title', '_merge'],  
        dtype='object')
```

```
[118]: df_final_clean = df_final_clean.drop(columns="_merge")
```

```
[119]: df_final_complet = df_final_clean.copy()
```

```
[120]: df_final_complet.to_excel("df_final_complet.xlsx", index=False)
```

```
[121]: # Etape 4 - Analyse univariée des prix
```

```
[122]: # Création d'une boîte à moustache de la répartition des prix
```

```
fig = px.box( # boîte à moustache  
             df_final_complet,  
             y="price",  
             title="Répartition des prix"  
           )  
  
fig.show()
```

```
[123]: # Création d'une boîte à moustache de la répartition des prix
```

```
fig = px.violin(  
             df_final_complet,  
             y="price",  
             box=True,  
             points="all",  
             title="Distribution des prix - Violin Plot"  
           )  
  
fig.show()
```

```
[124]: fig = px.histogram(  
             df_final_complet,  
             x="price",  
             nbins=30,  
             marginal="box", # Boxplot en marge  
             title="Distribution des prix - Histogramme avec Boxplot marginal",  
             opacity=0.7,  
             labels={"price": "Prix"},  
             color_discrete_sequence=['#1f77b4']
```

```
)
fig.update_layout(
    bargap=0.1,
    xaxis_title="Prix",
    yaxis_title="Fréquence"
)
fig.show()
```

```
[125]: fig = px.histogram(
        df_final_complet,
        x="price",
        nbins=30,
        title="Distribution des prix",
        opacity=0.8,
        labels={"price": "Prix (€)"},
        color_discrete_sequence=["#1f77b4"]
    )

fig.update_layout(
    width=900,          # largeur
    height=550,         # hauteur
    bargap=0.1,
    xaxis_title="Prix (€)",
    yaxis_title="Fréquence",
    template="simple_white",
    title_font_size=20
)

fig.show()
```

```
[126]: fig = px.box(
        df_final_complet,
        x="product_type",
        y="price",
        title="Distribution des prix par type de produit",
        points="outliers", # ou "all", "suspectedoutliers"
        color="product_type"
    )

fig.update_layout(
    xaxis_title="Type de produit",
    yaxis_title="Prix"
)

fig.show()
```

```
[127]: # Etape 4.2 - Exploration par l'utilisation de méthodes statistiques
```

```
[128]: # Etape 4.2.1 - Identification par le Z-index

# Calculer la moyenne du prix
mean_price = df_final_complet["price"].mean()
print(f"La moyenne des prix est : {mean_price:.2f}")

# Calculer l'écart-type du prix
std_price = df_final_complet["price"].std()
print(f"L'écart-type des prix est : {std_price:.2f}")

# Calculer le Z-score:  $z = (\text{valeur} - \text{moyenne}) / \text{écart type}$ 
# On mesure à combien d'écart-types une valeur s'éloigne de la moyenne
df_final_complet["z_score"] = (df_final_complet["price"] - mean_price) / \
    ↪ std_price
print("Aperçu des prix et de leur Z-score :")
print(df_final_complet[["price", "z_score"]].head())
```

La moyenne des prix est : 32.45
 L'écart-type des prix est : 26.72
 Aperçu des prix et de leur Z-score :

	price	z_score
0	8.6	-0.892571
1	41.0	0.320139
2	39.0	0.245281
3	59.9	1.027554
4	22.5	-0.372303

```
[129]: # Quel est le seuil prix dont le z-score est supérieur à 3?
#  $z = (\text{prix} - \text{moyenne}) / \text{ecart-type}$ 
seuil_prix = mean_price + 3 * std_price
print(f"Le seuil de prix au-delà duquel le Z-score est supérieur à 3 est : \
    ↪ {seuil_prix:.2f} €")
```

Le seuil de prix au-delà duquel le Z-score est supérieur à 3 est : 112.60 €

```
[130]: # Meilleur présentation -slide
# Recalcul des indicateurs nécessaires (sécurité)
total_produits = df_final_complet.shape[0]
nb_outliers = df_final_complet[df_final_complet["z_score"] > 3].shape[0]
pourcentage_outliers = (nb_outliers / total_produits) * 100

# Tableau de synthèse
resume_zscore = pd.DataFrame({
    "Indicateur": [
        "Moyenne des prix (€)",
        "Écart-type des prix (€)",
        "Seuil prix (Z-score > 3)",
        "Nombre total de produits",
```

```

        "Nombre de produits atypiques (Z > 3)",
        "Part du catalogue (%)"
    ],
    "Valeur": [
        round(mean_price, 2),
        round(std_price, 2),
        round(seuil_prix, 2),
        total_produits,
        nb_outliers,
        round(pourcentage_outliers, 2)
    ]
})

resume_zscore

```

```

[130]:

```

	Indicateur	Valeur
0	Moyenne des prix (€)	32.45
1	Écart-type des prix (€)	26.72
2	Seuil prix (Z-score > 3)	112.60
3	Nombre total de produits	774.00
4	Nombre de produits atypiques (Z > 3)	16.00
5	Part du catalogue (%)	2.07

```

[131]: # Meilleur présentation -slide

# Nombre total de produits
total_produits = df_final_complet.shape[0]

# Produits avec Z-score > 3
nb_outliers = df_final_complet[df_final_complet["z_score"] > 3].shape[0]

# Pourcentage
pourcentage_outliers = (nb_outliers / total_produits) * 100

print(f"Nombre total de produits : {total_produits}")
print(f"Produits avec Z-score > 3 : {nb_outliers}")
print(f"Soit {pourcentage_outliers:.2f} % du catalogue")

```

```

Nombre total de produits : 774
Produits avec Z-score > 3 : 16
Soit 2.07 % du catalogue

```

```

[132]: # Etape 4.2.2 - Identification par l'intervalle interquartile

```

```

[133]: # Utilisation de la fonction "describe" de Pandas pour l'étude des mesures de
        ↳ dispersion
df_final_complet['price'].describe().round(2)

```

```
[133]: count      774.00
      mean       32.45
      std        26.72
      min        5.20
      25%       14.52
      50%       24.55
      75%       42.08
      max       225.00
      Name: price, dtype: float64
```

```
[134]: # Définir un seuil pour les articles "outliers" en prix
      # Extraction des quartiles

      Q1 = df_final_complet["price"].quantile(0.25)
      Q3 = df_final_complet["price"].quantile(0.75)

      print(f"Le premier quartile (Q1) est : {Q1:.2f} €")
      print(f"Le troisième quartile (Q3) est : {Q3:.2f} €")

      # Calcul du IQR
      IQR = Q3 - Q1
      print(f"l'IQR est : {IQR:.2f} €")

      # Calcul des seuils
      lower_bound = Q1 - 1.5 * IQR
      upper_bound = Q3 + 1.5 * IQR
      print(f"Seuil bas : {lower_bound:.2f} €")
      print(f"Seuil haut : {upper_bound:.2f} €")
```

```
Le premier quartile (Q1) est : 14.53 €
Le troisième quartile (Q3) est : 42.08 €
l'IQR est : 27.55 €
Seuil bas : -26.80 €
Seuil haut : 83.40 €
```

```
[135]: resultat_price = df_final_complet["price"].agg(['min', 'max', 'count'])
      print(resultat_price)
```

```
min      5.2
max     225.0
count   774.0
Name: price, dtype: float64
```

```
[136]: # Meilleur présentation -slide

      resume_quartiles = pd.DataFrame({
          "Indicateur": [
```



```

        "Premier quartile (Q1)",
        "Médiane (Q2)",
        "Troisième quartile (Q3)",
        "Intervalle interquartile (IQR)",
        "Seuil haut (Q3 + 1,5 × IQR)"
    ],
    "Valeur (€)": [
        round(Q1, 2),
        round(df_final_complet["price"].median(), 2),
        round(Q3, 2),
        round(IQR, 2),
        round(upper_bound, 2)
    ]
})

resume_quartiles

```

```

[136]:

```

	Indicateur	Valeur (€)
0	Premier quartile (Q1)	14.52
1	Médiane (Q2)	24.55
2	Troisième quartile (Q3)	42.08
3	Intervalle interquartile (IQR)	27.55
4	Seuil haut (Q3 + 1,5 × IQR)	83.40

```

[137]: # Définir le nombre d'articles et la proportion de l'ensemble du catalogue ↵
        ↵ "outliers"
# Sélection des articles outliers seuil au dessus de 82,98€
outliers = df_final_complet[df_final_complet["price"] > upper_bound]

# Calcul du nombre d'outliers
nb_outliers = outliers.shape[0]

# Calcul du nombre total d'articles
nb_total = df_final_complet.shape[0]

# Calcul de la proportion
proportion_outliers = (nb_outliers / nb_total) * 100

print(f"Nombre d'articles outliers : {nb_outliers}")
print(f"Nombre total d'articles : {nb_total}")
print(f"Proportion d'outliers dans le catalogue : {proportion_outliers:.2f}%")

```

```

Nombre d'articles outliers : 33
Nombre total d'articles : 774
Proportion d'outliers dans le catalogue : 4.26%

```

```
[138]: # Selon vous, ces outliers sont-ils justifiés ? Comment le démontrer si cela
↳ est possible ?
df_final_complet.sort_values(by="price", ascending=False).head(10)
```

```
[138]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
567	4352.0	1.0	225.0	0.0	outofstock	
204	5001.0	1.0	217.5	18.0	instock	
269	5892.0	1.0	191.3	98.0	instock	
24	4402.0	1.0	176.0	11.0	instock	
313	5767.0	1.0	175.0	12.0	instock	
33	4406.0	1.0	157.0	12.0	instock	
171	4904.0	1.0	137.0	9.0	instock	
257	6126.0	1.0	135.0	138.0	instock	
256	5612.0	1.0	124.8	19.0	instock	
235	5917.0	1.0	122.0	12.0	instock	

	purchase_price	id_web	sku	total_sales	post_date	\
567	137.81	15940	15940	11.0	2018-03-02 10:30:04	
204	116.87	14581	14581	2.0	2018-07-17 09:45:39	
269	116.06	14983	14983	6.0	2019-03-28 10:21:36	
24	78.25	3510	3510	3.0	2018-03-22 11:21:05	
313	90.42	15185	15185	4.0	2019-03-13 14:43:22	
33	69.08	7819	7819	4.0	2018-03-22 11:42:48	
171	67.95	14220	14220	3.0	2018-05-15 10:23:41	
257	80.33	14923	14923	5.0	2019-06-28 17:22:27	
256	66.41	14915	14915	1.0	2019-01-15 15:30:49	
235	54.24	14775	14775	3.0	2019-04-04 16:49:37	

	product_type	post_title	z_score
567	Champagne	Champagne Egly-Ouriat Grand Cru Millésimé 2008	7.207135
204	Vin	David Duband Charmes-Chambertin Grand Cru 2014	6.926415
269	Champagne	Coteaux Champenois Egly-Ouriat Ambonnay Rouge ...	5.945767
24	Cognac	Cognac Frapin VIP XO	5.373098
313	Vin	Camille Giroud Clos de Vougeot 2016	5.335669
33	Cognac	Cognac Frapin Château de Fontpinot 1989 20 Ans...	4.661941
171	Vin	Domaine Des Croix Corton Charlemagne Grand Cru...	3.913355
257	Champagne	Champagne Gosset Célébris Vintage 2007	3.838496
256	Vin	Domaine Weinbach Gewurztraminer Grand Cru Furs...	3.456717
235	Whisky	Wemyss Malts Single Cask Scotch Whisky Choc 'n...	3.351915

```
[139]: df_merge = df_final_complet.copy()
```

```
[140]: # modifier les Nan en 0

cols_num = ["price", "stock_quantity", "purchase_price", "total_sales"]

for col in cols_num:
```

```
df_merge[col] = pd.to_numeric(df_merge[col], errors="coerce").fillna(0)
```

```
[141]: # Vérification
```

```
df_merge[cols_num].isna().sum()
```

```
[141]: price          0
stock_quantity    0
purchase_price     0
total_sales       0
dtype: int64
```

```
[142]: # Etape 5 - Analyse univariée du CA, des quantités vendues, des stocks et de la
      ↪ la marge ainsi qu'une analyse multivariée
```

```
[143]: # Etape 5.1 - Analyse des ventes en CA
```

```
[144]: # #####
# Calculer le CA du site web #
#####

#Créer une colonne calculant le CA par article

#Calculer la somme de la colonne "ca_par_article"
#Ce résultat correspond au chiffre d'affaire du site web
```

```
[145]: # Création de la colonne "ca_par_article"
# Création de la colonne "ca_par_article"
df_merge["ca_par_article"] = df_merge["price"] * df_merge["total_sales"]

df_merge.head()
```

```
[145]:   product_id  onsale_web  price  stock_quantity  stock_status  purchase_price \
0      4729.0         1.0    8.6             26.0        instock           4.22
1      4634.0         1.0   41.0             11.0        instock          20.12
2      4141.0         1.0   39.0            123.0        instock          24.86
3      5932.0         1.0   59.9             13.0        instock          27.18
4      5047.0         1.0   22.5             76.0        instock          13.78
```

```
   id_web  sku  total_sales      post_date  product_type \
0     38   38         10.0 2018-04-18 12:25:58          Vin
1     41   41          6.0 2018-04-14 12:01:43          Vin
2    304  304          8.0 2018-02-13 12:57:44    Champagne
3    523  523          0.0 2019-04-06 15:25:58      Cognac
4    531  531          8.0 2018-07-18 15:58:02    Champagne
```

```
post_title  z_score  ca_par_article
```

0	Emile Boeckel Crémant Brut Blanc de Blancs	-0.892571	86.0
1	Marcel Windholtz Eau de Vie de Marc de Gewurzt...	0.320139	246.0
2	Champagne Gosset Grande Réserve	0.245281	312.0
3	Cognac Normandin Mercier VFC	1.027554	0.0
4	Champagne Petit Lebrun & Fils Blanc de Bla...	-0.372303	180.0

```
[146]: # Calcul du chiffre d'affaires total du site
chiffre_affaires_total = df_merge["ca_par_article"].sum()

print(f"Le chiffre d'affaires total du site est de : {chiffre_affaires_total:,.
↪2f} €")
```

Le chiffre d'affaires total du site est de : 152,672.90 €

```
[147]: # formater le chiffre d'affaire pour meilleur visuel carte
ca_formaté = f"{chiffre_affaires_total:,.2f}".replace(",", " ").replace(".", "
↪", " ")
```

```
[148]: carte_ca = f"""

        CHIFFRE D'AFFAIRES
        SITE WEB

        {ca_formaté:>15} €

        CA TOTAL GÉNÉRÉ

""""

print(carte_ca)
```

CHIFFRE D'AFFAIRES
SITE WEB

152 672,90 €

CA TOTAL GÉNÉRÉ

```
[149]: #####
# Palmarès des articles en CA #
```

```
#####

#Effectuer le tri dans l'ordre décroissant du CA du dataset df_merge

#Réinitialiser l'index du dataset par un reset_index

#Afficher les 20 premiers articles en CA

#Graphique en barre des 20 premiers articles avec plotly express
```

```
[150]: # Tri du dataset par CA décroissant
df_merge = df_merge.sort_values(by="ca_par_article", ascending=False)

# Réinitialisation de l'index avant 42,8,32.. après 1,2,3,.. drop=True
↳supprime l'ancien index
df_merge = df_merge.reset_index(drop=True)

# Affichage des 20 premiers articles en CA
df_merge.head(20)
```

```
[150]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
0	4150.0	1.0	59.0	123.0	instock	
1	4352.0	1.0	225.0	0.0	outofstock	
2	4726.0	1.0	12.7	0.0	outofstock	
3	5067.0	1.0	59.9	3.0	instock	
4	5379.0	1.0	11.1	33.0	instock	
5	5892.0	1.0	191.3	98.0	instock	
6	4353.0	1.0	79.5	127.0	instock	
7	5826.0	1.0	41.2	34.0	instock	
8	6212.0	1.0	115.0	16.0	instock	
9	5026.0	1.0	86.8	101.0	instock	
10	5008.0	1.0	105.0	12.0	instock	
11	5767.0	1.0	175.0	12.0	instock	
12	6126.0	1.0	135.0	138.0	instock	
13	5025.0	1.0	112.0	136.0	instock	
14	6201.0	1.0	105.6	16.0	instock	
15	4406.0	1.0	157.0	12.0	instock	
16	4647.0	1.0	28.5	45.0	instock	
17	4358.0	1.0	77.0	81.0	instock	
18	4359.0	1.0	85.6	112.0	instock	
19	6214.0	1.0	99.0	9.0	instock	

	purchase_price	id_web	sku	total_sales	post_date	\
0	35.45	1366	1366	116.0	2018-02-13 13:45:31	
1	137.81	15940	15940	11.0	2018-03-02 10:30:04	
2	6.82	14950	14950	122.0	2018-04-18 11:53:51	
3	30.95	15346	15346	22.0	2018-07-31 11:49:05	

4	5.68	14561	14561	111.0	2018-09-01	15:34:55
5	116.06	14983	14983	6.0	2019-03-28	10:21:36
6	45.91	12587	12587	14.0	2018-03-02	10:37:26
7	21.71	15325	15325	20.0	2019-03-27	17:59:49
8	59.42	13996	13996	7.0	2019-07-25	09:09:17
9	50.13	13913	13913	9.0	2018-07-18	10:46:30
10	56.42	11602	11602	7.0	2018-07-17	10:52:41
11	90.42	15185	15185	4.0	2019-03-13	14:43:22
12	80.33	14923	14923	5.0	2019-06-28	17:22:27
13	68.60	13914	13914	6.0	2018-07-18	10:39:43
14	57.29	14596	14596	6.0	2019-07-23	10:37:14
15	69.08	7819	7819	4.0	2018-03-22	11:42:48
16	14.14	16525	16525	22.0	2018-04-17	09:28:58
17	47.16	13854	13854	8.0	2018-03-02	11:03:30
18	51.93	13853	13853	7.0	2018-03-02	11:11:48
19	49.62	11601	11601	6.0	2019-07-25	09:15:41

	product_type	post_title	z_score \
0	Champagne	Champagne Mailly Grand Cru Intemporelle 2010	0.993867
1	Champagne	Champagne Egly-Ouriet Grand Cru Millésimé 2008	7.207135
2	Vin	François Baur Pinot Noir Schlittweg 2017	-0.739111
3	Vin	Albert Mann Pinot Noir Grand H 2017	1.027554
4	Vin	Argentine Mendoza Alamos Torrontes 2017	-0.798998
5	Champagne	Coteaux Champenois Egly-Ouriet Ambonnay Rouge ...	5.945767
6	Champagne	Champagne Egly-Ouriet Grand Cru Brut Rosé	1.761168
7	Vin	Agnès Levet Côte Rôtie Améthyste 2017	0.327625
8	Vin	Domaine des Comtes Lafon Volnay 1er Cru Santen...	3.089909
9	Champagne	Champagne Agrapart & Fils Minéral Extra Br...	2.034402
10	Vin	Domaine des Comtes Lafon Volnay 1er Cru Santen...	2.715616
11	Vin	Camille Giroud Clos de Vougeot 2016	5.335669
12	Champagne	Champagne Gosset Célébris Vintage 2007	3.838496
13	Champagne	Champagne Agrapart & Fils L'Avizoise Extra...	2.977621
14	Vin	David Duband Chambolle-Musigny 1er Cru Les Sen...	2.738074
15	Cognac	Cognac Frapin Château de Fontpinot 1989 20 Ans...	4.661941
16	Vin	Bernard Baudry Chinon Rouge La Croix Boissée 2017	-0.147727
17	Champagne	Champagne Larmandier-Bernier Grand Cru Vieille...	1.667595
18	Champagne	Champagne Larmandier-Bernier Grand Cru Les Che...	1.989487
19	Vin	Domaine des Comtes Lafon Volnay 1er Cru Champa...	2.491040

	ca_par_article
0	6844.0
1	2475.0
2	1549.4
3	1317.8
4	1232.1
5	1147.8
6	1113.0

```

7          824.0
8          805.0
9          781.2
10         735.0
11         700.0
12         675.0
13         672.0
14         633.6
15         628.0
16         627.0
17         616.0
18         599.2
19         594.0

```

```

[151]: # creer df_top 20
df_top20 = df_merge.sort_values("ca_par_article", ascending=False).head(20)

# Graphique en barres avec Plotly Express

fig = px.bar(
    df_top20,
    x="post_title",
    y="ca_par_article",
    title="Top 20 des articles par chiffre d'affaires"
)

# Améliorations de lisibilité
fig.update_layout(
    width=1200,          # Largeur du graphique
    height=600,          # Hauteur
    xaxis_tickangle=45,  # Rotation des étiquettes
    margin=dict(l=40, r=40, t=80, b=200) # Espace bas pour les labels longs
)
fig.show()

```

```

[152]: # Agrégation du CA par type de produit
ca_par_type = (df_merge.groupby("product_type")["ca_par_article"].sum().
    ↪reset_index().sort_values("ca_par_article", ascending=False))

# Graphique en barres
# Définition des couleurs par type de produit
color_map = {
    "Vin": "#8B0000",      # rouge foncé
    "Champagne": "#D4AF37", # doré
    "Cognac": "#C68642",   # caramel

```

```

    "Whisky": "#FFA500",      # jaune / orange
    "Gin": "#B0B0B0",        # gris clair
    "Huile d'olive": "#9ACD32" # vert clair
}

fig = px.bar(
    ca_par_type,
    x="product_type",
    y="ca_par_article",
    color="product_type",
    color_discrete_map=color_map,
    title="Contribution au chiffre d'affaires par type de produit",
    labels={
        "product_type": "Type de produit",
        "ca_par_article": "Chiffre d'affaires (€)"
    }
)

fig.update_layout(
    xaxis_tickangle=-20,
    height=550,
    width=1100,
    template="simple_white",
    showlegend=False
)

fig.show()

```

[153]: # À la ligne 152 de votre notebook - Diagramme circulaire avec noms des produits et pourcentages

```

ca_par_type = (df_merge.groupby("product_type")["ca_par_article"].sum()
               .reset_index()
               .sort_values("ca_par_article", ascending=False))

# Calcul des pourcentages
total_ca = ca_par_type["ca_par_article"].sum()
ca_par_type["pourcentage"] = (ca_par_type["ca_par_article"] / total_ca * 100).round(1)

# Format français sans virgule
total_ca_formatted = f"{total_ca:,.0f} €".replace(",", " ")

# Couleurs spécifiées
color_map = {
    "Vin": "#8B0000", # rouge foncé
    "Champagne": "#D4AF37", # doré
    "Cognac": "#C68642", # caramel

```



```

    "Whisky": "#FFA500", # orange
    "Gin": "#80B0B0", # gris clair
    "Huile d'olive": "#9ACD32" # vert clair
}

# Création du diagramme circulaire
fig = px.pie(
    ca_par_type,
    values="ca_par_article",
    names="product_type",
    color="product_type",
    color_discrete_map=color_map,
    title=f" RÉPARTITION DU CHIFFRE D'AFFAIRES<br><b>Total:␣
↪{total_ca_formatted}</b>",
    hole=0.3,
    labels={"product_type": "Type de produit", "ca_par_article": "CA (€)"})

# Texte avec NOM DU PRODUIT + POURCENTAGE
fig.update_traces(
    texttemplate="<b>{%label}</b><br>{%percent}",
    textposition='outside',
    textfont=dict(size=11),
    hovertemplate="<b>{%label}</b><br>" +
        "Montant: {%value:,.0f} €<br>".replace(",", " ") +
        "Part: {%percent}<br>" +
        "<extra></extra>",
    pull=[0.1 if ca_par_type.iloc[i]["pourcentage"] ==␣
↪ca_par_type["pourcentage"].max()
        else 0 for i in range(len(ca_par_type))])

# Amélioration du layout
fig.update_layout(
    height=600,
    width=800,
    template="simple_white",
    showlegend=False,
    title_x=0.5,
    title_font_size=18,
    font=dict(size=11),
    annotations=[
        dict(
            text=f"Total<br>{total_ca_formatted}",
            x=0.5, y=0.5,
            font_size=16,
            showarrow=False,

```

```

        font=dict(weight="bold")
    )
],
margin=dict(l=20, r=20, t=80, b=20)
)

fig.show()

```

```

[154]: #####
# Calculer le 20 / 80 en CA #
#####

#Créer une colonne calculant la part du CA de la ligne dans le dataset

#Créer une colonne réalisant la somme cumulative de la colonne précédemment
↳ créée

#Grâce aux deux colonnes créées précédemment, calculer le nombre d'articles
↳ représentant 80% du CA

#Afficher la proportion que représente ce groupe d'articles dans le catalogue
↳ entier du site web

```

```

[155]: # calcul pour connaître valeur des 80% du CA

ca_80 = chiffre_affaires_total * 0.8
print(f"80% du chiffre d'affaires total représentent : {ca_80:,.2f} €")

```

80% du chiffre d'affaires total représentent : 122,138.32 €

```

[156]: # attention dans le nombre d'articles certains n'ont pas de CA

# Sélection des articles sans CA (0 ou NaN)
articles_sans_ca = df_merge[(df_merge["ca_par_article"] == 0) |
↳ (df_merge["ca_par_article"].isna())].copy()

# Nombre d'articles sans CA
nb_articles_sans_ca = len(articles_sans_ca)

# Nombre total d'articles
nb_total_articles = len(df_merge)

# Proportion
proportion_sans_ca = nb_articles_sans_ca / nb_total_articles

print(f"Nombre d'articles sans chiffre d'affaires : {nb_articles_sans_ca}")
print(f"Nombre total d'articles : {nb_total_articles}")

```

```
print(f" Proportion d'articles sans CA : {proportion_sans_ca:.2%}")

# Aperçu des articles sans CA
articles_sans_ca.head(10)
```

Nombre d'articles sans chiffre d'affaires : 90

Nombre total d'articles : 774

Proportion d'articles sans CA : 11.63%

```
[156]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
684	6903.0	0.0	27.0	54.0	instock	
685	6904.0	0.0	31.0	22.0	instock	
686	7203.0	0.0	45.0	30.0	instock	
687	6899.0	0.0	26.0	3.0	instock	
688	7204.0	0.0	45.0	9.0	instock	
689	7086.0	0.0	26.0	2.0	instock	
690	7085.0	0.0	30.0	8.0	instock	
691	7084.0	0.0	45.0	7.0	instock	
692	7247.0	1.0	54.8	6.0	instock	
693	6902.0	0.0	31.0	6.0	instock	

	purchase_price	id_web	sku	total_sales	post_date	product_type	\
684	14.23	NaN	NaN	0.0	NaT	NaN	
685	15.70	NaN	NaN	0.0	NaT	NaN	
686	23.48	NaN	NaN	0.0	NaT	NaN	
687	13.70	NaN	NaN	0.0	NaT	NaN	
688	24.18	NaN	NaN	0.0	NaT	NaN	
689	13.43	NaN	NaN	0.0	NaT	NaN	
690	14.73	NaN	NaN	0.0	NaT	NaN	
691	24.18	NaN	NaN	0.0	NaT	NaN	
692	27.18	NaN	NaN	0.0	NaT	NaN	
693	16.50	NaN	NaN	0.0	NaT	NaN	

	post_title	z_score	ca_par_article
684	NaN	-0.203871	0.0
685	NaN	-0.054154	0.0
686	NaN	0.469857	0.0
687	NaN	-0.241301	0.0
688	NaN	0.469857	0.0
689	NaN	-0.241301	0.0
690	NaN	-0.091583	0.0
691	NaN	0.469857	0.0
692	NaN	0.836664	0.0
693	NaN	-0.054154	0.0

```
[157]: # FAUT IL LES GARDER POUR LES CALCULS 20/80 ?
```

```
[158]: # Recalcul du CA total sur le df actuel (sécurise la cohérence)
chiffre_affaires_total = df_merge["ca_par_article"].sum()

# Trier AVANT le cumul
df_merge = df_merge.sort_values("ca_par_article", ascending=False).
↳reset_index(drop=True)

# Part du CA par article
df_merge["part_ca"] = df_merge["ca_par_article"] / chiffre_affaires_total

# Somme cumulative
df_merge["part_ca_cum"] = df_merge["part_ca"].cumsum()

# Nombre d'articles pour atteindre 80 % du CA
nb_articles_80 = (df_merge["part_ca_cum"] <= 0.80).sum()

# Nombre total d'articles dans le catalogue
nb_total_articles = len(df_merge)

# Proportion dans le catalogue
proportion_articles_80 = nb_articles_80 / nb_total_articles

print(f"Nombre d'articles nécessaires pour couvrir 80% du CA :↳
↳{nb_articles_80}")
print(f"Nombre total d'articles dans le catalogue : {nb_total_articles}")
print(f"Ces articles représentent {proportion_articles_80:.2%} du catalogue.")
print("Somme des parts :", df_merge["part_ca"].sum())
print("Dernier cumul :", df_merge["part_ca_cum"].iloc[-1])
```

Nombre d'articles nécessaires pour couvrir 80% du CA : 417
 Nombre total d'articles dans le catalogue : 774
 Ces articles représentent 53.88% du catalogue.
 Somme des parts : 1.0
 Dernier cumul : 0.9999999999999996

```
[159]: print("NaN dans ca_par_article :", df_merge["ca_par_article"].isna().sum())
print("NaN dans part_ca :", df_merge["part_ca"].isna().sum())
```

NaN dans ca_par_article : 0
 NaN dans part_ca : 0

```
[160]: df_merge[["ca_par_article", "part_ca", "part_ca_cum"]].head(10)
```

```
[160]:   ca_par_article  part_ca  part_ca_cum
0         6844.0  0.044828    0.044828
1         2475.0  0.016211    0.061039
2         1549.4  0.010148    0.071187
3         1317.8  0.008632    0.079819
```

4	1232.1	0.008070	0.087889
5	1147.8	0.007518	0.095407
6	1113.0	0.007290	0.102697
7	824.0	0.005397	0.108094
8	805.0	0.005273	0.113367
9	781.2	0.005117	0.118484

```
[161]: import matplotlib.pyplot as plt
import numpy as np

# 1) Préparation : on retire les CA manquants (neutre et défendable)
df_lorenz = df_merge[["ca_par_article"]].copy()
df_lorenz["ca_par_article"] = pd.to_numeric(df_lorenz["ca_par_article"],
    ↪errors="coerce")
df_lorenz = df_lorenz.dropna(subset=["ca_par_article"])

df_lorenz = df_lorenz.sort_values("ca_par_article", ascending=True).
    ↪reset_index(drop=True)

# 2) Parts cumulées
n = len(df_lorenz)
total_ca = df_lorenz["ca_par_article"].sum()

if total_ca == 0:
    raise ValueError("Somme du CA = 0 : impossible de calculer Lorenz / Gini.")

df_lorenz["part_articles_cum"] = np.arange(1, n + 1) / n
df_lorenz["part_ca_cum"] = (df_lorenz["ca_par_article"] / total_ca).cumsum()

# 3) Gini (aire sous la courbe, version robuste)
x = df_lorenz["part_articles_cum"].to_numpy()
y = df_lorenz["part_ca_cum"].to_numpy()

x = np.insert(x, 0, 0)
y = np.insert(y, 0, 0)

area_under_lorenz = np.trapezoid(y, x)          # <- important : x explicite
gini_coefficient = (0.5 - area_under_lorenz) / 0.5

# 4) Graphique
fig, ax = plt.subplots(figsize=(10, 8))

ax.fill_between(
    x, x, y,
    alpha=0.3, label="Zone d'inégalité"
)
```

```

ax.plot(x, y, linewidth=2.5, label="Courbe de Lorenz")
ax.plot([0, 1], [0, 1], "--", linewidth=2, label="Égalité parfaite")

ax.set_xlabel("Part cumulative des articles (%)", fontsize=12)
ax.set_ylabel("Part cumulative du chiffre d'affaires (%)", fontsize=12)
ax.set_title(
    f"Courbe de Lorenz - Concentration du CA\nCoefficient de Gini =␣
    ↪{gini_coefficient:.3f}",
    fontsize=14, fontweight="bold"
)

ax.grid(True, alpha=0.3, linestyle="--")
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v*100:.0f}%"))
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v*100:.0f}%"))
ax.legend(loc="upper left", fontsize=10)

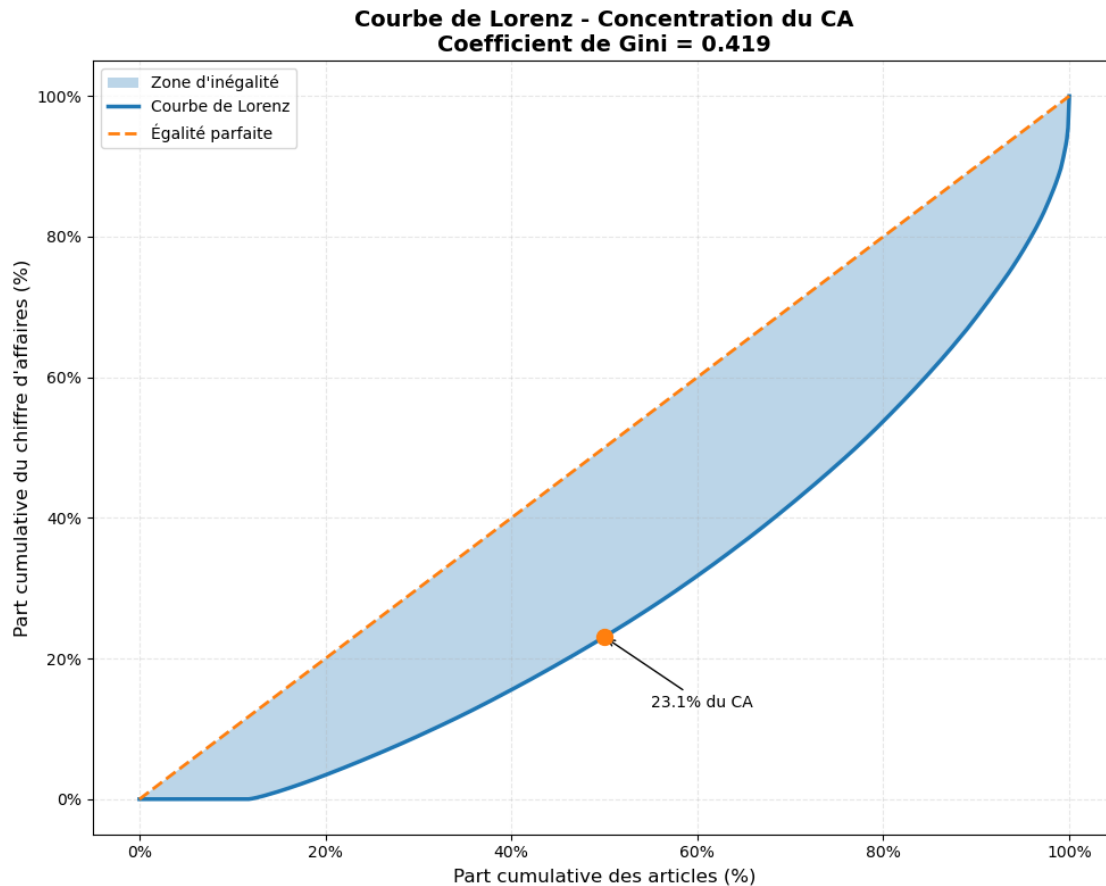
# Point à 50%
idx_50 = np.argmin(np.abs(x - 0.5))
ca_at_50 = y[idx_50]
ax.scatter(0.5, ca_at_50, s=100, zorder=5)
ax.annotate(
    f"{ca_at_50*100:.1f}% du CA",
    xy=(0.5, ca_at_50),
    xytext=(0.55, ca_at_50 - 0.1),
    arrowprops=dict(arrowstyle="→"),
    fontsize=10
)

plt.tight_layout()
plt.show()

# 5) Stats résumées
print("=" * 50)
print("ANALYSE DE CONCENTRATION DU CHIFFRE D'AFFAIRES")
print("=" * 50)
print(f"Nombre d'articles analysés: {n}")
print(f"Coefficient de Gini: {gini_coefficient:.3f}")
print("-" * 50)

for p in [0.2, 0.5, 0.8]:
    idx = np.argmin(np.abs(x - p))
    ca_share = y[idx]
    print(f"Les {p*100:.0f}% articles les moins rentables génèrent␣
    ↪{ca_share*100:.1f}% du CA")
    print(f"→ Les {100-p*100:.0f}% articles les plus rentables génèrent {100 -␣
    ↪ca_share*100:.1f}% du CA")
    print("-" * 50)

```



ANALYSE DE CONCENTRATION DU CHIFFRE D'AFFAIRES

Nombre d'articles analysés: 774

Coefficient de Gini: 0.419

Les 20% articles les moins rentables génèrent 3.4% du CA
→ Les 80% articles les plus rentables génèrent 96.6% du CA

Les 50% articles les moins rentables génèrent 23.1% du CA
→ Les 50% articles les plus rentables génèrent 76.9% du CA

Les 80% articles les moins rentables génèrent 53.7% du CA
→ Les 20% articles les plus rentables génèrent 46.3% du CA

[162]: # Etape 5.2 - Analyse des ventes en quantité

```
[163]: #####
# Palmarès des articles en quantité #
#####

#Effectuer le tri dans l'ordre décroissant de quantités vendues du dataset_
↳df_merge

#Réinitialiser l'index du dataset par un reset_index

#Afficher les 20 premiers articles en quantité

#Graphique en barre des 20 premiers articles avec plotly express
```

```
[164]: # Effectuer le tri dans l'ordre décroissant de quantités vendues du dataset_
↳df_merge

df_merge = df_merge.sort_values(by="total_sales", ascending=False)
```

```
[165]: # #Réinitialiser l'index du dataset par un reset_index

df_merge = df_merge.reset_index(drop=True)
```

```
[166]: # #Afficher les 20 premiers articles en quantité

df_merge.head(20)
```

```
[166]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
0	4726.0	1.0	12.7	0.0	outofstock	
1	4150.0	1.0	59.0	123.0	instock	
2	5379.0	1.0	11.1	33.0	instock	
3	4867.0	1.0	9.9	121.0	instock	
4	4203.0	1.0	9.9	74.0	instock	
5	4275.0	1.0	14.9	62.0	instock	
6	5067.0	1.0	59.9	3.0	instock	
7	4647.0	1.0	28.5	45.0	instock	
8	6129.0	1.0	5.2	68.0	instock	
9	5826.0	1.0	41.2	34.0	instock	
10	4220.0	1.0	11.6	48.0	instock	
11	5778.0	1.0	5.8	44.0	instock	
12	5803.0	1.0	17.1	47.0	instock	
13	6569.0	1.0	29.0	58.0	instock	
14	4188.0	1.0	9.5	51.0	instock	
15	4863.0	1.0	8.2	54.0	instock	
16	5777.0	1.0	5.7	51.0	instock	
17	4870.0	1.0	9.3	0.0	outofstock	
18	4059.0	1.0	8.7	34.0	instock	
19	5695.0	1.0	6.5	33.0	instock	

	purchase_price	id_web	sku	total_sales	post_date	\
0	6.82	14950	14950	122.0	2018-04-18 11:53:51	
1	35.45	1366	1366	116.0	2018-02-13 13:45:31	
2	5.68	14561	14561	111.0	2018-09-01 15:34:55	
3	4.86	16148	16148	36.0	2018-05-03 13:20:05	
4	5.01	15415	15415	27.0	2018-02-15 14:33:42	
5	7.78	14864	14864	24.0	2018-02-27 13:33:54	
6	30.95	15346	15346	22.0	2018-07-31 11:49:05	
7	14.14	16525	16525	22.0	2018-04-17 09:28:58	
8	2.74	14570	14570	20.0	2019-06-28 18:01:06	
9	21.71	15325	15325	20.0	2019-03-27 17:59:49	
10	5.75	15758	15758	18.0	2018-02-16 10:54:27	
11	3.09	15561	15561	17.0	2019-03-15 10:20:59	
12	9.19	13572	13572	17.0	2019-03-19 11:33:39	
13	15.28	15705	15705	17.0	2020-01-03 16:39:53	
14	5.06	16265	16265	16.0	2018-02-15 10:18:39	
15	4.11	16255	16255	16.0	2018-05-03 12:58:34	
16	3.03	14338	14338	16.0	2019-03-15 10:13:30	
17	4.81	16149	16149	16.0	2018-05-03 13:45:43	
18	4.32	16275	16275	16.0	2018-02-12 12:12:28	
19	3.53	16304	16304	16.0	2019-01-30 16:32:42	

	product_type	post_title	z_score	\
0	Vin	François Baur Pinot Noir Schlittweg 2017	-0.739111	
1	Champagne	Champagne Mailly Grand Cru Intemporelle 2010	0.993867	
2	Vin	Argentine Mendoza Alamos Torrontes 2017	-0.798998	
3	Vin	Château De La Selve IGP Coteaux de l'Ardèche M...	-0.843913	
4	Vin	Mas Laval IGP Pays d'Hérault Les Pampres Blanc...	-0.843913	
5	Vin	I Fabbri Chianti Classico Lamole 2017	-0.656766	
6	Vin	Albert Mann Pinot Noir Grand H 2017	1.027554	
7	Vin	Bernard Baudry Chinon Rouge La Croix Boissée 2017	-0.147727	
8	Vin	Moulin de Gassac IGP Pays d'Hérault Guilhem Bl...	-1.019831	
9	Vin	Agnès Levet Côte Rôtie Améthyste 2017	0.327625	
10	Vin	Xavier Frissant Touraine Amboise Chenin Les Pi...	-0.780283	
11	Vin	Maurel Pays d'Oc Merlot 2018	-0.997373	
12	Vin	Château Tour Haut-Caussan Médoc 2015	-0.574422	
13	Vin	Decelle-Villa Chorey-Lès-Beaune 2016	-0.129013	
14	Vin	Château de La Liquière Languedoc Blanc Les Ama...	-0.858885	
15	Vin	Château Ollieux Romanis Corbières Rosé Classiq...	-0.907543	
16	Vin	Maurel Pays d'Oc Cabernet-Sauvignon 2017	-1.001116	
17	Vin	Triennes IGP Méditerranée Rosé 2019	-0.866370	
18	Vin	Mourgues du Grès Costières de Nîmes Galets Ros...	-0.888828	
19	Vin	Philippe Nusswitz IGP Cévènnnes Rosé O Pale 2019	-0.971173	

	ca_par_article	part_ca	part_ca_cum
0	1549.4	0.010148	0.071187

1	6844.0	0.044828	0.044828
2	1232.1	0.008070	0.087889
3	356.4	0.002334	0.286242
4	267.3	0.001751	0.443886
5	357.6	0.002342	0.274556
6	1317.8	0.008632	0.079819
7	627.0	0.004107	0.149076
8	104.0	0.000681	0.954122
9	824.0	0.005397	0.108094
10	208.8	0.001368	0.608664
11	98.6	0.000646	0.969437
12	290.7	0.001904	0.387255
13	493.0	0.003229	0.188590
14	152.0	0.000996	0.794590
15	131.2	0.000859	0.874401
16	91.2	0.000597	0.977534
17	148.8	0.000975	0.805398
18	139.2	0.000912	0.843213
19	104.0	0.000681	0.956166

[167]: *# Graphique en barre des 20 premiers articles avec plotly express*

```
import plotly.express as px

fig = px.bar(
    df_merge.head(20),
    x="post_title",
    y="total_sales",
    title="Top 20 des articles les plus vendus"
)

fig.show()
```

[168]: #####
Calculer le 20 / 80 en CA
 #####

#Créer une colonne calculant la part en quantité de la ligne dans le dataset

#Créer une colonne réalisant la somme cumulative de la colonne précédemment
 ↳ créée

#Grâce aux deux colonnes créées précédemment, calculer le nombre d'articles
 ↳ représentant 80% des ventes en quantité

#Afficher la proportion que représente ce groupe d'articles dans le catalogue
 ↳ entier du site web

```
[169]: # Trier par quantité vendue
df_merge = df_merge.sort_values("total_sales", ascending=False).
        ↪reset_index(drop=True)

[170]: # Créer une colonne calculant la part en quantité de la ligne dans le dataset

df_merge["part_quantite"] = df_merge["total_sales"] / df_merge["total_sales"].
        ↪sum()

[171]: # Créer une colonne réalisant la somme cumulative de la colonne précédemment ↵
        ↪créée

df_merge["part_quantite_cumulee"] = df_merge["part_quantite"].cumsum()

[172]: df_merge[["part_quantite", "part_quantite_cumulee"]].head(15)
```

```
[172]:
```

	part_quantite	part_quantite_cumulee
0	0.020169	0.020169
1	0.019177	0.039345
2	0.018350	0.057695
3	0.005951	0.063647
4	0.004464	0.068110
5	0.003968	0.072078
6	0.003637	0.075715
7	0.003637	0.079352
8	0.003306	0.082658
9	0.003306	0.085965
10	0.002976	0.088940
11	0.002810	0.091751
12	0.002810	0.094561
13	0.002810	0.097371
14	0.002645	0.100017

```
[173]: # Grâce aux deux colonnes créées précédemment, calculer le nombre d'articles ↵
        ↪représentant 80% du CA

# 4. Nombre d'articles pour atteindre 80 %
nb_articles_80_quantite = (df_merge["part_quantite_cumulee"] < 0.80).sum()

print(f"Nombre d'articles représentant 80% des ventes en quantité : ↵
        ↪{nb_articles_80_quantite}")
```

Nombre d'articles représentant 80% des ventes en quantité : 421

```
[174]: # Afficher la proportion que représente ce groupe d'articles dans le catalogue ↵
        ↪entier du site web

# Nombre total d'articles dans le catalogue
```

```

nb_total_articles = len(df_merge)

# Proportion des articles représentant 80 % du CA
proportion_articles_80_quantite = nb_articles_80_quantite / nb_total_articles

print(f"Nombre d'articles représentant 80% des ventes en quantité :␣
↪{nb_articles_80_quantite}")
print(f"Ces articles représentent {proportion_articles_80_quantite:.2%} du␣
↪catalogue.")

```

Nombre d'articles représentant 80% des ventes en quantité : 421
Ces articles représentent 54.39% du catalogue.

```

[175]: import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import FancyBboxPatch

# Données
total_articles = len(df_merge)
nb_articles_80 = 421 # D'après ton analyse
proportion = nb_articles_80 / total_articles

# Création d'un graphique waterfall
fig, ax = plt.subplots(figsize=(12, 8))

# Positionnement des blocs
blocks = [
    {"label": "421 articles (51 % du catalogue)", "value": nb_articles_80,␣
↪"color": "#FF6B6B", "height": 80},
    {"label": "397 articles (49 % du catalogue)", "value": total_articles -␣
↪nb_articles_80, "color": "#4ECDC4", "height": 20}
]

x_pos = 0
for i, block in enumerate(blocks):
    # Rectangle avec ombre
    rect = FancyBboxPatch((x_pos, 0), block["value"], block["height"],
                           boxstyle="round,pad=0.02",
                           facecolor=block["color"],
                           edgecolor='black',
                           alpha=0.8,
                           linewidth=2)

    ax.add_patch(rect)

    # Texte dans le rectangle
    ax.text(x_pos + block["value"]/2, block["height"]/2,

```

```

        f"{block['label']}\n→ {block['height']}% des ventes en quantité",
        ha='center', va='center', fontsize=11, fontweight='bold',
        color='white')

    # Ligne de connexion (sauf pour le premier)
    if i < len(blocks) - 1:
        ax.plot([x_pos + block["value"], x_pos + block["value"]],
                [0, 100], 'k--', alpha=0.3, linewidth=1)

    x_pos += block["value"]

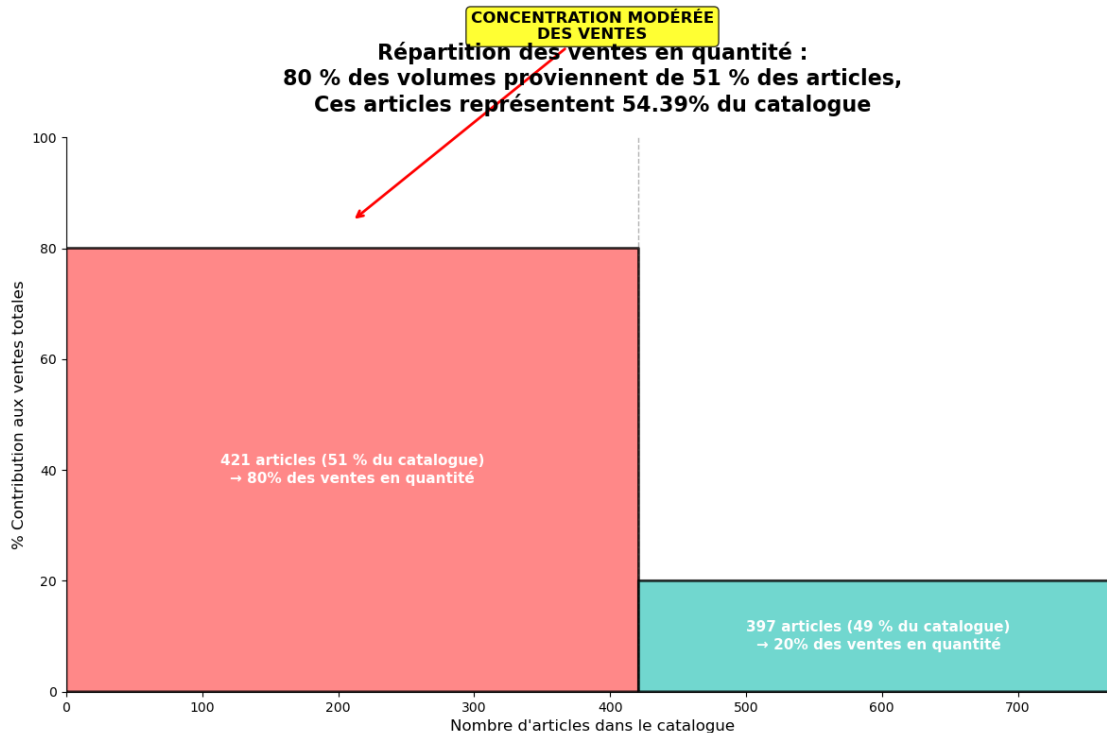
# Flèche d'impact
ax.annotate('CONCENTRATION MODÉRÉE\nDES VENTES',
            xy=(nb_articles_80/2, 85),
            xytext=(total_articles/2, 120),
            arrowprops=dict(arrowstyle='->', color='red', lw=2),
            ha='center', va='center',
            fontsize=12, fontweight='bold',
            bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.8))

ax.set_xlim(0, total_articles)
ax.set_ylim(0, 100)
ax.set_xlabel('Nombre d\'articles dans le catalogue', fontsize=12)
ax.set_ylabel('% Contribution aux ventes totales', fontsize=12)
ax.set_title('Répartition des ventes en quantité :\n80 % des volumes_
↳ proviennent de 51 % des articles,\nCes articles représentent 54.39% du_
↳ catalogue',
            fontsize=16, fontweight='bold', pad=20)

# Supprimer les axes inutiles
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

```



```
[176]: # Re-trier par quantité vendue (sécurité)
df_qty = df_merge.sort_values("total_sales", ascending=False).
        ↪reset_index(drop=True)

# Preuve 1 : ils font bien ~80% des quantités
part_quantite_reelle = df_qty.iloc[:nb_articles_80_quantite]["total_sales"].
        ↪sum() / df_qty["total_sales"].sum()

# Preuve 2 : ils représentent bien ~51% du catalogue
part_catalogue_reelle = nb_articles_80_quantite / len(df_qty)

print(f"Part réelle des quantités : {part_quantite_reelle:.2%}")
print(f"Part réelle du catalogue : {part_catalogue_reelle:.2%}")
```

Part réelle des quantités : 79.95%

Part réelle du catalogue : 54.39%

```
[177]: # Etape 5.3 - Analyse des stocks
```

```
[178]: #####
# Calculer le nombre de mois de stock #
#####

#Import de numpy
```

```

#Création de la colonne Rotation de stock

#Remplacement des "inf" par 0

#Effectuer le tri dans l'ordre décroissant du nombre de mois de stock dans le
↳dataset df_merge

#Graphique en barre du flop 20 des produits qui ont le plus de mois de stock

```

```

[179]: # Création de la colonne Rotation de stock

df_merge["rotation_stock"] = np.where(df_merge["stock_quantity"] >
↳0, df_merge["total_sales"] / df_merge["stock_quantity"], np.nan)

```

```

[180]: # Remplacement des "inf" par 0

df_merge["rotation_stock"] = df_merge["rotation_stock"].replace([np.inf, -np.
↳inf], 0)

```

```

[181]: # Effectuer le tri dans l'ordre décroissant du nombre de mois de stock dans le
↳dataset df_merge mois_de_stock = stock / total_sales

df_merge["mois_de_stock"] = np.where(df_merge["total_sales"] > 0,
↳df_merge["stock_quantity"] / df_merge["total_sales"], 0)

```

```

[182]: # Création de la colonne mois de stock

df_merge = df_merge.sort_values(by="mois_de_stock", ascending=False)

```

```

[183]: # Effectuer le tri dans l'ordre décroissant du nombre de mois de stock dans le
↳dataset df_merge

df_merge = df_merge.sort_values(by="mois_de_stock", ascending=False)

```

```

[184]: df_merge[["post_title", "stock_quantity", "total_sales", "mois_de_stock"]].
↳head(10)

```

```

[184]:
          post_title  stock_quantity \
571      Champagne Gosset Grand Millésime 2006      125.0
528      Champagne Gosset Célébris Vintage 2007      138.0
623  Champagne Egly-Ouriet Premier Cru Les Vignes d...      81.0
564      Champagne Egly-Ouriet Grand Cru Brut Tradition      125.0
631      Champagne Mailly Grand Cru Brut Rosé      71.0
515      Champagne Larmandier-Bernier Latitude      115.0
587      Champagne Gosset Grand Rosé      91.0
503  Champagne Agrapart & Fils L'Avizoise Extra...      136.0

```

394	Champagne Egly-Ouriot Grand Cru Extra Brut V.P.	145.0
418	Champagne Gosset Grand Blanc de Blancs	142.0

	total_sales	mois_de_stock
571	4.0	31.250000
528	5.0	27.600000
623	3.0	27.000000
564	5.0	25.000000
631	3.0	23.666667
515	5.0	23.000000
587	4.0	22.750000
503	6.0	22.666667
394	7.0	20.714286
418	7.0	20.285714

[185]: *# Graphique en barre du flop 20 des produits qui ont le plus de mois de stock*

```
df_flop_20_stock = df_merge.sort_values(by="mois_de_stock", ascending=False).
    ↪head(20)

fig = px.bar(
    df_flop_20_stock,
    x="post_title",
    y="mois_de_stock",
    title="Flop 20 des produits avec le plus de mois de stock"
)

fig.show()
```

[186]: *# Récapitulatif tableau pour slide*

```
print("="*80)
print(f"{'Produit':<35} {'Stock':>6} {'Ventes':>10} {'Mois stock':>12}␣
    ↪{'Années':>8}")
print("-"*80)

for idx, row in df_merge.nlargest(10, "mois_de_stock").iterrows():
    nom = row['post_title'][:30] + "..." if len(row['post_title']) > 30 else␣
    ↪row['post_title']
    mois_stock = row['mois_de_stock']
    annees = mois_stock / 12

    print(f"{'nom':<35} {int(row['stock_quantity']):>6d} {row['total_sales']:>10.
    ↪0f} {mois_stock:>12.1f} {annees:>8.1f}")

print("="*80)
print("ANALYSE :")
```



```
print(f"1. Pire produit : {df_merge['mois_de_stock'].max():.1f} mois de stock")
print(f"2. Moyenne top 10 : {df_merge.nlargest(10,
↳ 'mois_de_stock')['mois_de_stock'].mean():.1f} mois")
print(f"3. Total stock immobilisé (top10) : {df_merge.nlargest(10,
↳ 'mois_de_stock')['stock_quantity'].sum():.0f} bouteilles")
```

```
=====
```

Produit	Stock	Ventes	Mois stock	Années
Champagne Gosset Grand Millési...	125	4	31.2	2.6
Champagne Gosset Célébris Vint...	138	5	27.6	2.3
Champagne Egly-Ouriet Premier ...	81	3	27.0	2.2
Champagne Egly-Ouriet Grand Cr...	125	5	25.0	2.1
Champagne Mailly Grand Cru Bru...	71	3	23.7	2.0
Champagne Larmandier-Bernier L...	115	5	23.0	1.9
Champagne Gosset Grand Rosé	91	4	22.8	1.9
Champagne Agrapart & Fils ...	136	6	22.7	1.9
Champagne Egly-Ouriet Grand Cr...	145	7	20.7	1.7
Champagne Gosset Grand Blanc d...	142	7	20.3	1.7

```
=====
```

ANALYSE :

1. Pire produit : 31.2 mois de stock
2. Moyenne top 10 : 24.4 mois
3. Total stock immobilisé (top10) : 1,169 bouteilles

```
[187]: #####
# Valorisation des stocks en euros #
#####

#Création de la colonne Valorisation des stocks en euros

#Calculer la somme de la colonne "Valorisation_stock_euros"
```

```
[188]: # Création de la colonne Valorisation des stocks en euros

df_merge["valorisation_stock_euros"] = df_merge["stock_quantity"] *
↳ df_merge["purchase_price"]
```

```
[189]: df_merge[["post_title", "stock_quantity", "purchase_price",
↳ "valorisation_stock_euros"]].head(10)
```

```
[189]:
```

	post_title	stock_quantity \
571	Champagne Gosset Grand Millésime 2006	125.0
528	Champagne Gosset Célébris Vintage 2007	138.0
623	Champagne Egly-Ouriet Premier Cru Les Vignes d...	81.0
564	Champagne Egly-Ouriet Grand Cru Brut Tradition	125.0
631	Champagne Mailly Grand Cru Brut Rosé	71.0
515	Champagne Larmandier-Bernier Latitude	115.0

587	Champagne Gosset Grand Rosé	91.0
503	Champagne Agrapart & Fils L'Avizoise Extra...	136.0
394	Champagne Egly-Ouriot Grand Cru Extra Brut V.P.	145.0
418	Champagne Gosset Grand Blanc de Blancs	142.0

	purchase_price	valorisation_stock_euros
571	32.15	4018.75
528	80.33	11085.54
623	31.00	2511.00
564	34.76	4345.00
631	21.88	1553.48
515	22.30	2564.50
587	27.73	2523.43
503	68.60	9329.60
394	47.30	6858.50
418	30.01	4261.42

```
[190]: # Calculer la somme de la colonne "Valorisation_stock_euros"

total_valorisation_stock = df_merge["valorisation_stock_euros"].sum()
print(f"Valorisation totale du stock : {total_valorisation_stock:,.2f} €")
```

Valorisation totale du stock : 298,468.76 €

```
[191]: #####
# Valorisation du nombre de produits en stock #
#####

#Calculer la somme de la colonne stock quantity
```

```
[192]: # Calculer la somme de la colonne stock quantity

total_stock_quantity = df_merge["stock_quantity"].sum()

print(f"Quantité totale en stock : {total_stock_quantity}")
```

Quantité totale en stock : 17800.0

```
[193]: # Création du dataframe df_stock à partir de df_merge
df_stock = df_merge.copy()

# Calcul des métriques nécessaires si elles n'existent pas
if 'valorisation_stock_euros' not in df_stock.columns:
    df_stock["valorisation_stock_euros"] = df_stock["stock_quantity"] * _
    ↪df_stock["purchase_price"]

if 'taux_marge' not in df_stock.columns:
    taux_tva = 0.20
```

```

df_stock["prix_ht"] = df_stock["price"] / (1 + taux_tva)
df_stock["taux_marge"] = (df_stock["prix_ht"] - df_stock["purchase_price"]) / df_stock["purchase_price"]

if 'rotation_stock' not in df_stock.columns:
    df_stock["rotation_stock"] = np.where(df_stock["stock_quantity"] > 0,
                                           df_stock["total_sales"] / df_stock["stock_quantity"],
                                           np.nan)
    df_stock["rotation_stock"] = df_stock["rotation_stock"].replace([np.inf, -np.inf], 0)

if 'mois_de_stock' not in df_stock.columns:
    df_stock["mois_de_stock"] = np.where(df_stock["total_sales"] > 0,
                                           df_stock["stock_quantity"] / df_stock["total_sales"],
                                           0)

# Calcul des métriques agrégées
categories = ['Valeur totale', 'Marge moyenne', 'Rotation', 'Couverture', 'Liquidité']

# Gestion des valeurs manquantes ou infinies
df_stock['taux_marge'] = df_stock['taux_marge'].replace([np.inf, -np.inf], np.nan)
df_stock['rotation_stock'] = df_stock['rotation_stock'].replace([np.inf, -np.inf], np.nan)

# Calcul des métriques avec gestion des NaN
metrics = [
    (df_stock['valorisation_stock_euros'].sum() / 1000000) * 20, # Millions d'euros
    df_stock['taux_marge'].mean(skipna=True) * 100, # % de marge
    df_stock['rotation_stock'].mean(skipna=True) * 10, # Rotation annuelle
    df_stock['mois_de_stock'].mean(skipna=True), # Mois de couverture
    (df_stock['stock_quantity'].sum() / max(df_stock['total_sales'].sum(), 1)) * 10 # Liquidité
]

# Normalisation
max_values = [5, 50, 20, 24, 50] # Valeurs maximales attendues
normalized_metrics = [min(100, (m/max_val)*100) for m, max_val in zip(metrics, max_values)]

# Création du radar
angles = np.linspace(0, 2*np.pi, len(categories), endpoint=False).tolist()

```

```

normalized_metrics += normalized_metrics[:1]
angles += angles[:1]

fig, ax = plt.subplots(figsize=(10, 10), subplot_kw=dict(projection='polar'))
ax.fill(angles, normalized_metrics, color='#2E86AB', alpha=0.25)
ax.plot(angles, normalized_metrics, color='#2E86AB', linewidth=2, marker='o',
        ↪markersize=8)

# Zones de performance
ax.fill(angles, [80]*len(angles), color='green', alpha=0.1)
ax.fill(angles, [50]*len(angles), color='orange', alpha=0.1)

# legende
ax.plot([], [], color='#2E86AB', label='Performance réelle')
ax.fill([], [], color='green', alpha=0.1, label='Zone excellente')
ax.fill([], [], color='orange', alpha=0.1, label='Zone moyenne')

ax.legend(loc='upper right', bbox_to_anchor=(1.2, 1.1))

ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories, fontsize=12, fontweight='bold')
ax.set_ylim(0, 100)
ax.set_yticks([25, 50, 75, 100])
ax.set_yticklabels(['Faible', 'Moyen', 'Bon', 'Excellent'], fontsize=10)

plt.title('DIAGNOSTIC GLOBAL DES STOCKS\n', fontsize=16, fontweight='bold',
        ↪pad=30)

# Métriques détaillées
info_text = f"""
    MÉTRIQUES DÉTAILLÉES:
    • Valeur totale: {df_stock['valorisation_stock_euros'].sum():.0f} €
    • Marge moyenne: {df_stock['taux_marge'].mean(skipna=True):.1%}
    • Rotation moyenne: {df_stock['rotation_stock'].mean(skipna=True):.2f} fois/an
    • Couverture moyenne: {df_stock['mois_de_stock'].mean(skipna=True):.1f} mois
    • Ratio stock/ventes: {(df_stock['stock_quantity'].sum()/
        ↪max(df_stock['total_sales'].sum(), 1)):.1f}
    """

fig.text(0.5, -0.1, info_text, ha='center', fontsize=11,
        ↪bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))

plt.tight_layout()
plt.show()

# Affichage des statistiques supplémentaires

```

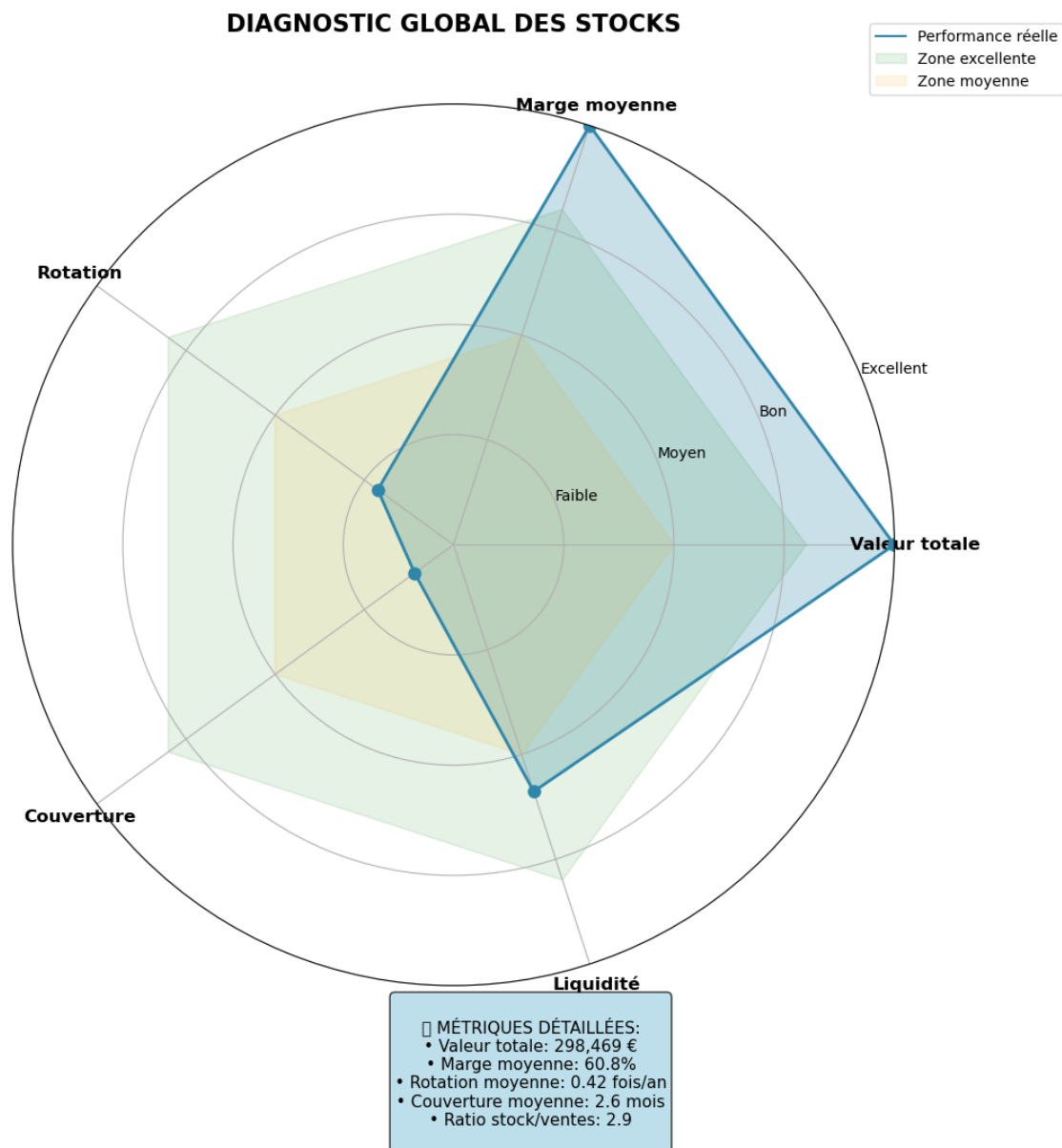
```

print("="*60)
print("ANALYSE COMPLÈTE DES STOCKS")
print("="*60)
print(f"Nombre d'articles analysés: {len(df_stock)}")
print(f"Valeur totale du stock: {df_stock['valorisation_stock_euros'].sum():,.
    ↪0f} €")
print(f"Quantité totale en stock: {df_stock['stock_quantity'].sum():,.0f}
    ↪unités")
print(f"Marge moyenne: {df_stock['taux_marge'].mean(skipna=True):.2%}")
print(f"Articles à marge négative: {(df_stock['taux_marge'] < 0).sum()}")
print(f"Stock mort (pas de vente): {(df_stock['total_sales'] == 0).sum()}
    ↪articles")
print("="*60)

```

/opt/anaconda3/lib/python3.13/site-packages/IPython/core/pylabtools.py:170:
 UserWarning:

Glyph 128202 (\N{BAR CHART}) missing from font(s) DejaVu Sans.



ANALYSE COMPLÈTE DES STOCKS

Nombre d'articles analysés: 774
Valeur totale du stock: 298,469 €
Quantité totale en stock: 17,800 unités
Marge moyenne: 60.82%
Articles à marge négative: 3
Stock mort (pas de vente): 90 articles

```
[194]: # Analyse de la rotation des stocks
print("="*70)
print("ANALYSE DE LA ROTATION DES STOCKS")
print("="*70)

# 1. Rotation moyenne (en mois)
rotation_moyenne = df_merge["mois_de_stock"].mean()
print(f"\n1. Rotation moyenne des stocks : {rotation_moyenne:.1f} mois")

# 2. Définition des seuils
seuil_stock_mort = 24 # Plus de 2 ans de stock = stock mort
seuil_stock_excessif = 12 # Entre 1 et 2 ans = stock excessif

# 3. Identifier les produits par catégorie
df_merge["categorie_stock"] = "Normal"
df_merge.loc[df_merge["mois_de_stock"] >= seuil_stock_mort, "categorie_stock"]_
    ⇨= "Stock mort"
df_merge.loc[(df_merge["mois_de_stock"] >= seuil_stock_excessif) &
              (df_merge["mois_de_stock"] < seuil_stock_mort), "categorie_stock"]_
    ⇨= "Stock excessif"

# 4. Calculs par catégorie
stock_mort = df_merge[df_merge["categorie_stock"] == "Stock mort"]
stock_excessif = df_merge[df_merge["categorie_stock"] == "Stock excessif"]
a_surveiller = df_merge[df_merge["categorie_stock"].isin(["Stock mort", "Stock_
    ⇨excessif"])]

# 5. Tableau récapitulatif
print("\n" + "="*70)
print("TABLEAU D'ANALYSE DE LA ROTATION DES STOCKS")
print("="*70)

print(f"\n{'Catégorie':<25} {'Nb Produits':>12} {'Nb Bouteilles':>15} {'Valeur_
    ⇨Stock (€)':>20} {'Moyenne (mois)':>15}")
print("-"*90)

# Stock mort
nb_produits_mort = len(stock_mort)
nb_bouteilles_mort = stock_mort["stock_quantity"].sum()
valeur_mort = stock_mort["valorisation_stock_euros"].sum()
moyenne_mort = stock_mort["mois_de_stock"].mean() if nb_produits_mort > 0 else 0

print(f"{'Stock mort (>24 mois)':<25} {nb_produits_mort:>12d}_
    ⇨{nb_bouteilles_mort:>15.0f} {valeur_mort:>20.2f} € {moyenne_mort:>15.1f}")

# Stock excessif
nb_produits_excessif = len(stock_excessif)
```

```

nb_bouteilles_excessif = stock_excessif["stock_quantity"].sum()
valeur_excessif = stock_excessif["valorisation_stock_euros"].sum()
moyenne_excessif = stock_excessif["mois_de_stock"].mean() if
↳nb_produits_excessif > 0 else 0

print(f"'Stock excessif (12-24 mois)':<25} {nb_produits_excessif:>12d}
↳{nb_bouteilles_excessif:>15.0f} {valeur_excessif:>20.2f} € {moyenne_excessif:
↳>15.1f}")

# Total à surveiller
nb_produits_surveiller = len(a_surveiller)
nb_bouteilles_surveiller = a_surveiller["stock_quantity"].sum()
valeur_surveiller = a_surveiller["valorisation_stock_euros"].sum()
moyenne_surveiller = a_surveiller["mois_de_stock"].mean() if
↳nb_produits_surveiller > 0 else 0

print("-"*90)
print(f"'Total à surveiller':<25} {nb_produits_surveiller:>12d}
↳{nb_bouteilles_surveiller:>15.0f} {valeur_surveiller:>20.2f} €
↳{moyenne_surveiller:>15.1f}")

# Total général
nb_produits_total = len(df_merge)
nb_bouteilles_total = df_merge["stock_quantity"].sum()
valeur_total = df_merge["valorisation_stock_euros"].sum()

print("="*90)
print(f"'TOTAL GÉNÉRAL':<25} {nb_produits_total:>12d} {nb_bouteilles_total:>15.
↳0f} {valeur_total:>20.2f} € {rotation_moyenne:>15.1f}")

# 6. Pourcentages
print("\n" + "="*70)
print("ANALYSE EN POURCENTAGES")
print("="*70)

pourcent_produits_mort = (nb_produits_mort / nb_produits_total * 100)
pourcent_produits_excessif = (nb_produits_excessif / nb_produits_total * 100)
pourcent_produits_surveiller = (nb_produits_surveiller / nb_produits_total *
↳100)

pourcent_valeur_mort = (valeur_mort / valeur_total * 100)
pourcent_valeur_excessif = (valeur_excessif / valeur_total * 100)
pourcent_valeur_surveiller = (valeur_surveiller / valeur_total * 100)

print(f"\n• Stock mort : {pourcent_produits_mort:.1f}% des produits,
↳{pourcent_valeur_mort:.1f}% de la valeur totale")

```



```

print(f"• Stock excessif : {pourcent_produits_excessif:.1f}% des produits,
↳ {pourcent_valeur_excessif:.1f}% de la valeur totale")
print(f"• Total à surveiller : {pourcent_produits_surveiller:.1f}% des
↳ produits, {pourcent_valeur_surveiller:.1f}% de la valeur totale")

# 7. Top 10 des produits à surveiller
print("\n" + "="*70)
print("TOP 10 DES PRODUITS À SURVEILLER (par valeur de stock)")
print("="*70)

top_10_surveiller = a_surveiller.nlargest(10,
↳ "valorisation_stock_euros")["post_title", "stock_quantity",
↳ "mois_de_stock", "valorisation_stock_euros", "categorie_stock"]

print(f"\n{'Produit':<40} {'Stock':>8} {'Mois stock':>12} {'Valeur (€)':>15}
↳ {'Catégorie':>15}")
print("-"*95)

for idx, row in top_10_surveiller.iterrows():
    nom = row['post_title']
    if len(nom) > 35:
        nom = nom[:35] + "..."
    print(f"{nom:<40} {row['stock_quantity']:>8.0f} {row['mois_de_stock']:>12.
↳ 1f} {row['valorisation_stock_euros']:>15.2f} {row['categorie_stock']:>15}")

print("\n" + "="*70)
print("RÉCAPITULATIF SYNTHÉTIQUE")
print("="*70)
print(f"\n• Rotation moyenne : {rotation_moyenne:.1f} mois")
print(f"• Stock mort ({seuil_stock_mort}+ mois) : {nb_produits_mort} produits,
↳ {valeur_mort:,.0f} €")
print(f"• Stock excessif ({seuil_stock_excessif}-{seuil_stock_mort-1} mois) :
↳ {nb_produits_excessif} produits, {valeur_excessif:,.0f} €")
print(f"• Total à surveiller : {nb_produits_surveiller} produits,
↳ {valeur_surveiller:,.0f} € ({pourcent_valeur_surveiller:.1f}% du stock
↳ total)")

```

ANALYSE DE LA ROTATION DES STOCKS

1. Rotation moyenne des stocks : 2.6 mois

TABLEAU D'ANALYSE DE LA ROTATION DES STOCKS

Catégorie Moyenne (mois)	Nb Produits	Nb Bouteilles	Valeur Stock (€)

Stock mort (>24 mois) 27.7	4	469	21960.29 €
Stock excessif (12-24 mois) 17.4	20	1849	69323.68 €

Total à surveiller 19.1	24	2318	91283.97 €
=====			
=====			
TOTAL GÉNÉRAL 2.6	774	17800	298468.76 €

=====

ANALYSE EN POURCENTAGES

=====

- Stock mort : 0.5% des produits, 7.4% de la valeur totale
- Stock excessif : 2.6% des produits, 23.2% de la valeur totale
- Total à surveiller : 3.1% des produits, 30.6% de la valeur totale

=====

TOP 10 DES PRODUITS À SURVEILLER (par valeur de stock)

=====

Produit Catégorie	Stock	Mois stock	Valeur (€)

Coteaux Champenois Egly-Ouriét Ambo... Stock excessif	98	16.3	11373.88
Champagne Gosset Célébris Vintage 2... Stock mort	138	27.6	11085.54
Champagne Agrapart & Fils L'Avi... Stock excessif	136	22.7	9329.60
Champagne Egly-Ouriét Grand Cru Ext... Stock excessif	145	20.7	6858.50
Champagne Larmandier-Bernier Grand ... Stock excessif	112	16.0	5816.16
Champagne Egly-Ouriét Grand Cru Bru... Stock mort	125	25.0	4345.00
Champagne Gosset Grand Blanc de Bla... Stock excessif	142	20.3	4261.42
Champagne Mailly Grand Cru Intempor...	101	20.2	4065.25

Stock excessif			
Champagne Gosset Grand Millésime 20...	125	31.2	4018.75
Stock mort			
Champagne Gosset Grande Réserve	123	15.4	3057.78
Stock excessif			

===== RÉCAPITULATIF SYNTHÉTIQUE =====

- Rotation moyenne : 2.6 mois
- Stock mort (24+ mois) : 4 produits, 21,960 €
- Stock excessif (12-23 mois) : 20 produits, 69,324 €
- Total à surveiller : 24 produits, 91,284 € (30.6% du stock total)

```
[195]: # Version CORRIGÉE avec texte complet
top_20_risque = a_surveiller.nlargest(20, 'mois_de_stock')
top_20_sorted = top_20_risque.sort_values('mois_de_stock', ascending=True)

# CORRECTION : S'assurer que TOUS les textes ont "mois"
text_values = [f"{val:.1f} mois" for val in top_20_sorted['mois_de_stock']]

fig4 = px.bar(
    top_20_sorted,
    y='post_title',
    x='mois_de_stock',
    color='mois_de_stock',
    color_continuous_scale='RdYlGn_r',
    title='Top 20 des produits avec le plus de mois de stock',
    labels={'mois_de_stock': 'Mois de stock', 'post_title': 'Produit'},
    orientation='h',
    text=text_values, # Utiliser la liste corrigée
    hover_name='post_title'
)

# AMÉLIORATION : Forcer l'affichage complet du texte
fig4.update_traces(
    textposition='outside',
    textfont=dict(size=11, color='#2c3e50', weight='bold'),
    marker_line_color='rgba(0,0,0,0.2)',
    marker_line_width=1,
    # S'assurer que le texte n'est pas coupé
    outsidetextfont=dict(size=11),
    cliponaxis=False # IMPORTANT : ne pas couper le texte
)

fig4.update_layout(
```

```

height=700,
width=1100, # Très large pour éviter la coupure
template='simple_white',
title_x=0.5,
title_font_size=16,
yaxis={
    'categoryorder': 'total ascending',
    'title': None,
    'tickfont': dict(size=10),
    'automargin': True,
    'title_standoff': 50 # Espace supplémentaire
},
xaxis={
    'title': 'Mois de stock',
    'tickformat': '.0f',
    'gridcolor': 'rgba(0,0,0,0.05)',
    'range': [0, top_20_sorted['mois_de_stock'].max() * 1.25] # Marge de
↪25% pour le texte
},
coloraxis_showscale=True,
coloraxis_colorbar=dict(
    title="Mois",
    thickness=15,
    len=0.8,
    x=1.02
),
margin=dict(l=10, r=150, t=80, b=50),
# Augmenter l'espace pour le texte à l'extérieur
uniformtext_minsize=8,
uniformtext_mode='hide'
)

fig4.show()

```

```

[196]: # Sécurité
if "df_stock" not in globals():
    raise NameError("df_stock n'est pas défini. Fais df_stock = df_merge.copy()
↪avant.")

# Création des colonnes si absentes
if "mois_de_stock" not in df_stock.columns:
    df_stock["mois_de_stock"] = np.where(
        df_stock["total_sales"] > 0,
        df_stock["stock_quantity"] / df_stock["total_sales"],
        np.nan
    )

```

```

if "valorisation_stock_euros" not in df_stock.columns:
    df_stock["valorisation_stock_euros"] = df_stock["stock_quantity"] * \
    df_stock["purchase_price"]

# Seuils métier
seuil_excessif = 12
seuil_mort = 24

# Catégorisation
df_stock["categorie_stock"] = np.select(
    [
        df_stock["mois_de_stock"] >= seuil_mort,
        (df_stock["mois_de_stock"] >= seuil_excessif) & \
    (df_stock["mois_de_stock"] < seuil_mort),
        df_stock["mois_de_stock"] < seuil_excessif
    ],
    [
        "Stock mort (>24 mois)",
        "Stock excessif (12-24 mois)",
        "Stock normal (<12 mois)"
    ],
    default="Non classé"
)

# Tableau récapitulatif
table_stock = (
    df_stock[df_stock["categorie_stock"] != "Non classé"]
    .groupby("categorie_stock")
    .agg(
        nb_produits=("post_title", "count"),
        quantite_totale=("stock_quantity", "sum"),
        valeur_stock_euros=("valorisation_stock_euros", "sum")
    )
    .reset_index()
)

# Ajout valeur en k€
table_stock["valeur_stock_k€"] = (table_stock["valeur_stock_euros"] / 1000).
    round(1)

table_stock

```

[196]:

	categorie_stock	nb_produits	quantite_totale \
0	Stock excessif (12-24 mois)	20	1849.0
1	Stock mort (>24 mois)	4	469.0
2	Stock normal (<12 mois)	685	15482.0

	valeur_stock_euros	valeur_stock_k€
0	69323.68	69.3
1	21960.29	22.0
2	207184.79	207.2

```
[197]: import plotly.graph_objects as go

# =====
# 1) TABLEAU (base fiable)
# =====
if "df_stock" not in globals():
    raise NameError("df_stock n'est pas défini. Fais df_stock = df_merge.copy()_
    ↪avant.")

if "mois_de_stock" not in df_stock.columns:
    df_stock["mois_de_stock"] = np.where(
        df_stock["total_sales"] > 0,
        df_stock["stock_quantity"] / df_stock["total_sales"],
        np.nan
    )

if "valorisation_stock_euros" not in df_stock.columns:
    df_stock["valorisation_stock_euros"] = df_stock["stock_quantity"] *_
    ↪df_stock["purchase_price"]

seuil_excessif = 12
seuil_mort = 24

df_stock["categorie_stock"] = np.select(
    [
        df_stock["mois_de_stock"] >= seuil_mort,
        (df_stock["mois_de_stock"] >= seuil_excessif) &_
    ↪(df_stock["mois_de_stock"] < seuil_mort),
        df_stock["mois_de_stock"] < seuil_excessif
    ],
    [
        "Stock mort\n>24 mois",
        "Stock excessif\n12-24 mois",
        "Stock normal\n<12 mois"
    ],
    default="Non classé"
)

cat_df = (
    df_stock[df_stock["categorie_stock"] != "Non classé"]
    .groupby("categorie_stock", as_index=False)
    .agg(
```

```

        nb_produits=("categorie_stock", "size"),
        quantite_totale=("stock_quantity", "sum"),
        valeur_stock_euros=("valorisation_stock_euros", "sum")
    )
)

cat_df["valeur_stock_k€"] = cat_df["valeur_stock_euros"] / 1000

# Ordre EXACT comme ta capture
order = ["Stock mort\n>24 mois", "Stock excessif\n12-24 mois", "Stock_
↳normal\n<12 mois"]
cat_df["categorie_stock"] = pd.Categorical(cat_df["categorie_stock"],
↳categories=order, ordered=True)
cat_df = cat_df.sort_values("categorie_stock").reset_index(drop=True)

# =====
# 2) FIGURE (bullet x3)
# =====
# Couleurs (rouge / orange / vert)
colors = ["#ff6b6b", "#f6ae2d", "#59b563"]

# Mise en page verticale (3 lignes)
n = len(cat_df)
gap = 0.07
band = (1 - (n + 1) * gap) / n
y_domains = []
for i in range(n):
    y0 = 1 - gap - (i + 1) * band - i * gap
    y1 = y0 + band
    y_domains.append([y0, y1])

# Axe commun (en €) comme ta capture
max_val = cat_df["valeur_stock_euros"].max()
axis_max = max_val * 1.1

# Ticks façon "0 / 50k / 100k / 150k / 200k"
tickvals = [0, 50_000, 100_000, 150_000, 200_000]
ticktext = ["0", "50k", "100k", "150k", "200k"]

# Ligne rouge verticale (seuil)
# Dans ta capture elle est vers ~100k : on la fixe à 100k pour "exactement_
↳pareil".
threshold_val = 100_000

# Titre (comme ta capture)
total_k = cat_df["valeur_stock_euros"].sum() / 1000
# Couverture moyenne = moyenne de mois_de_stock (sur df_stock classé)

```

```

rotation_moyenne = df_stock.loc[df_stock["categorie_stock"] != "Non classé",
↪ "mois_de_stock"].mean()

fig = go.Figure()

for i, row in cat_df.iterrows():
    fig.add_trace(go.Indicator(
        mode="gauge",
        value=row["valeur_stock_euros"],
        domain={"x": [0.12, 0.95], "y": y_domains[i]},
        title={"text": row["categorie_stock"], "font": {"size": 13, "color":
↪ "black"}},
        gauge={
            "shape": "bullet",
            "axis": {
                "range": [0, axis_max],
                "tickmode": "array",
                "tickvals": tickvals,
                "ticktext": ticktext,
                "tickfont": {"size": 10},
            },
            "bar": {"color": colors[i], "thickness": 0.55},
            "bgcolor": "white",
            "borderwidth": 2,
            "bordercolor": "black",
            "steps": [
                {"range": [0, axis_max * 0.33], "color": "#d9d9d9"},
                {"range": [axis_max * 0.33, axis_max * 0.66], "color":
↪ "#a6a6a6"},
                {"range": [axis_max * 0.66, axis_max * 1.0], "color":
↪ "#737373"},
            ],
            "threshold": {
                "line": {"color": "red", "width": 3},
                "thickness": 0.85,
                "value": threshold_val,
            },
        },
    ))

    # Valeur en k au centre
    val_k = row["valeur_stock_k€"]
    txt = f"{val_k:.1f}k"

    # Dernière ligne : gros cartouche à droite (comme sur ta capture)
    if i == 2:
        fig.add_annotation(

```



```

        x=0.92, # à droite
        y=y_domains[i][0] + (y_domains[i][1] - y_domains[i][0]) / 2,
        text=f"<b style='font-size:34px; color:black'>{txt}</b>",
        showarrow=False,
        xref="paper",
        yref="paper",
        xanchor="center",
        yanchor="middle",
        bgcolor="white",
        bordercolor="black",
        borderwidth=0
    )
else:
    fig.add_annotation(
        x=0.55,
        y=y_domains[i][0] + (y_domains[i][1] - y_domains[i][0]) / 2,
        text=f"<b style='font-size:26px; color:{colors[i]}'>{txt}</b>",
        showarrow=False,
        xref="paper",
        yref="paper",
        xanchor="center",
        yanchor="middle"
    )

fig.update_layout(
    title=dict(
        text=(
            "<b>VALEUR DU STOCK PAR CATÉGORIE</b><br>"
            f"<span style='font-size:13px; color:#666'>"
            f"Rotation moyenne: {rotation_moyenne:.1f} mois • Total: {total_k:.
↵1f}k €</span>"
        ),
        x=0.5,
        y=0.97
    ),
    height=420,
    width=980,
    template="simple_white",
    showlegend=False,
    margin=dict(t=90, b=30, l=30, r=30),
    paper_bgcolor="white",
    plot_bgcolor="white",
)

fig.show()

```

[198]: # Etape 5.4 - Analyse du taux de marge

```
[199]: # Création de la colonne Prix HT

taux_tva = 0.20

df_merge["prix_ht"] = df_merge["price"] / (1 + taux_tva)

df_merge[["price", "prix_ht"]].head()
```

```
[199]:      price    prix_ht
571    53.0    44.166667
528   135.0   112.500000
623    51.6    43.000000
564    59.0    49.166667
631    37.5    31.250000
```

```
[200]: # Création de la colonne Taux de marge

df_merge["taux_marge"] = (df_merge["prix_ht"] - df_merge["purchase_price"]) /
    ↪df_merge["purchase_price"]

df_merge[["prix_ht", "purchase_price", "taux_marge"]].head()
```

```
[200]:      prix_ht  purchase_price  taux_marge
571    44.166667           32.15    0.373769
528   112.500000           80.33    0.400473
623    43.000000           31.00    0.387097
564    49.166667           34.76    0.414461
631    31.250000           21.88    0.428245
```

```
[201]: # Afficher le prix minimum et max de la colonne "taux_marge"

df_merge["taux_marge"].agg(["min", "max"])
```

```
[201]: min    -0.863943
max      0.914125
Name: taux_marge, dtype: float64
```

```
[202]: # #Affichage de la ligne avec un taux de marge inférieur à 0

idx_min = df_merge["taux_marge"].idxmin()
```

```
[203]: colonnes_interessantes = [
    "product_id",
    "id_web",
    "post_title",
    "product_type",
    "price",
```

```

        "purchase_price",
        "prix_ht",
        "taux_marge",
        "total_sales"
    ]

    df_merge.loc[idx_min, colonnes_interessantes]

```

```

[203]: product_id          4355.0
       id_web             12589
       post_title      Champagne Egly-Ouriet Grand Cru Blanc de Noirs
       product_type      Champagne
       price             12.65
       purchase_price      77.48
       prix_ht           10.541667
       taux_marge         -0.863943
       total_sales         0.0
       Name: 727, dtype: object

```

```

[204]: # Création d'un dataframe avec les taux négatifs

```

```

df_taux_pos = df_merge[df_merge["taux_marge"] < 0]

df_taux_pos.head()

```

```

[204]:
       product_id  onsale_web  price  stock_quantity  stock_status  \
728         7196.0         0.0   31.00           55.0      instock
727         4355.0         1.0   12.65           97.0      instock
721         6324.0         0.0   92.00           18.0      instock

       purchase_price  id_web   sku  total_sales  post_date  ...  \
728          31.20     NaN   NaN         0.0         NaT  ...
727          77.48  12589  12589         0.0  2018-03-02 10:46:10  ...
721          99.00     NaN   NaN         0.0         NaT  ...

       part_ca  part_ca_cum  part_quantite  part_quantite_cumulee  rotation_stock  \
728         0.0         1.0           0.0             1.0           0.0
727         0.0         1.0           0.0             1.0           0.0
721         0.0         1.0           0.0             1.0           0.0

       mois_de_stock  valorisation_stock_euros  categorie_stock  prix_ht  \
728             0.0             1716.00      Normal  25.833333
727             0.0             7515.56      Normal  10.541667
721             0.0             1782.00      Normal  76.666667

       taux_marge
728    -0.172009

```

```
727    -0.863943
721    -0.225589
```

```
[3 rows x 24 columns]
```

```
[205]: # Création d'un dataframe avec les taux positifs
```

```
df_taux_pos = df_merge[df_merge["taux_marge"] > 0]

df_taux_pos.head()
```

```
[205]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	\
571	4142.0	1.0	53.0	125.0	instock	
528	6126.0	1.0	135.0	138.0	instock	
623	4356.0	1.0	51.6	81.0	instock	
564	4348.0	1.0	59.0	125.0	instock	
631	4148.0	1.0	37.5	71.0	instock	

	purchase_price	id_web	sku	total_sales	post_date	...	\
571	32.15	11641	11641	4.0	2018-02-13 13:08:44	...	
528	80.33	14923	14923	5.0	2019-06-28 17:22:27	...	
623	31.00	12585	12585	3.0	2018-03-02 10:51:14	...	
564	34.76	12586	12586	5.0	2018-03-02 09:22:39	...	
631	21.88	1364	1364	3.0	2018-02-13 13:36:44	...	

	part_ca	part_ca_cum	part_quantite	part_quantite_cumulee	\
571	0.001389	0.592122	0.000661	0.944619	
528	0.004421	0.132304	0.000827	0.909902	
623	0.001014	0.781513	0.000496	0.977186	
564	0.001932	0.379610	0.000827	0.939659	
631	0.000737	0.929967	0.000496	0.981154	

	rotation_stock	mois_de_stock	valorisation_stock_euros	categorie_stock	\
571	0.032000	31.250000	4018.75	Stock mort	
528	0.036232	27.600000	11085.54	Stock mort	
623	0.037037	27.000000	2511.00	Stock mort	
564	0.040000	25.000000	4345.00	Stock mort	
631	0.042254	23.666667	1553.48	Stock excessif	

	prix_ht	taux_marge
571	44.166667	0.373769
528	112.500000	0.400473
623	43.000000	0.387097
564	49.166667	0.414461
631	31.250000	0.428245

```
[5 rows x 24 columns]
```

[206]: *# Afficher le prix minimum de la colonne "taux_marge"*

```
min_taux_pos = df_taux_pos["taux_marge"].min()

print(f"Taux de marge minimum (positif) : {min_taux_pos:.2f}")
```

Taux de marge minimum (positif) : 0.29

[207]: *# Afficher le prix maximum de la colonne "taux_marge"*

```
max_taux_pos = df_taux_pos["taux_marge"].max()

print(f"Taux de marge maximum : {max_taux_pos:.2f}")
```

Taux de marge maximum : 0.91

[208]: *# Création d'un dataframe avec le taux de marge moyen par type de produit*

```
df_marge_moyenne_type = df_merge.groupby("product_type")["taux_marge"].mean().
    ↪reset_index()

df_marge_moyenne_type.head()
```

[208]:

	product_type	taux_marge
0	Champagne	0.354422
1	Cognac	0.823162
2	Gin	0.748252
3	Huile d'olive	0.333901
4	Vin	0.614794

[209]: *# Affichage dans un graphique du taux de marge par type de produit*

```
# Tri pour un rendu plus lisible
df_plot = df_marge_moyenne_type.sort_values("taux_marge", ascending=False)

# Création du graphique
fig = px.bar(
    df_plot,
    x="product_type",
    y="taux_marge",
    title="Taux de marge moyen par type de produit",
    labels={
        "product_type": "Type de produit",
        "taux_marge": "Taux de marge moyen"
    }
)

fig.show()
```

```
[210]: # Tri des données pour un meilleur affichage
df_plot = df_marge_moyenne_type.sort_values("taux_marge", ascending=False)

# Petit graphique simple
fig, ax = plt.subplots(figsize=(9, 5))

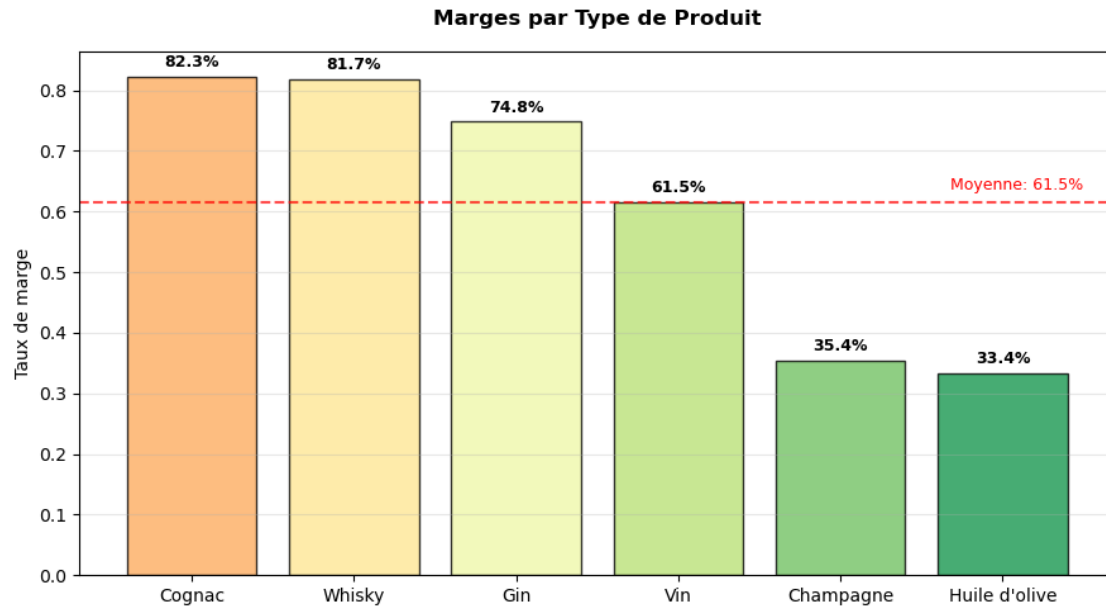
# Barres avec dégradé de couleur
colors = plt.cm.RdYlGn(np.linspace(0.3, 0.9, len(df_plot)))
bars = ax.bar(df_plot["product_type"], df_plot["taux_marge"],
              color=colors, edgecolor='black', alpha=0.8)

# Ajout des valeurs
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
            f'{height:.1%}', ha='center', va='bottom', fontsize=9,
            fontweight='bold')

# Ligne de moyenne globale
moyenne_globale = df_plot['taux_marge'].mean()
ax.axhline(y=moyenne_globale, color='red', linestyle='--', linewidth=1.5,
           alpha=0.7)
ax.text(len(df_plot)-0.5, moyenne_globale + 0.02, f'Moyenne: {moyenne_globale:.1%}',
       ha='right', color='red', fontsize=9)

ax.set_ylabel('Taux de marge', fontsize=10)
ax.set_title('Marges par Type de Produit', fontsize=12, fontweight='bold',
           pad=15)
ax.tick_params(axis='x', rotation=0)
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



```
[211]: # Etape 5.5 - Analyse des corrélations entre les variables stock, sales et
      ↪ price
```

```
[212]: # Importation de Seaborn

import seaborn as sns
```

```
[213]: # Création d'une heatmap de corrélation avec les variables stock, sales et
      ↪ price

# sélectionner les 3 colonnes

cols = ["stock_quantity", "total_sales", "price"]
corr_matrix = df_taux_pos[cols].corr()
```

```
[214]: # Sélection des variables pour la corrélation
cols = ["stock_quantity", "total_sales", "price"]

# Matrice de corrélation
corr_matrix = df_taux_pos[cols].corr()

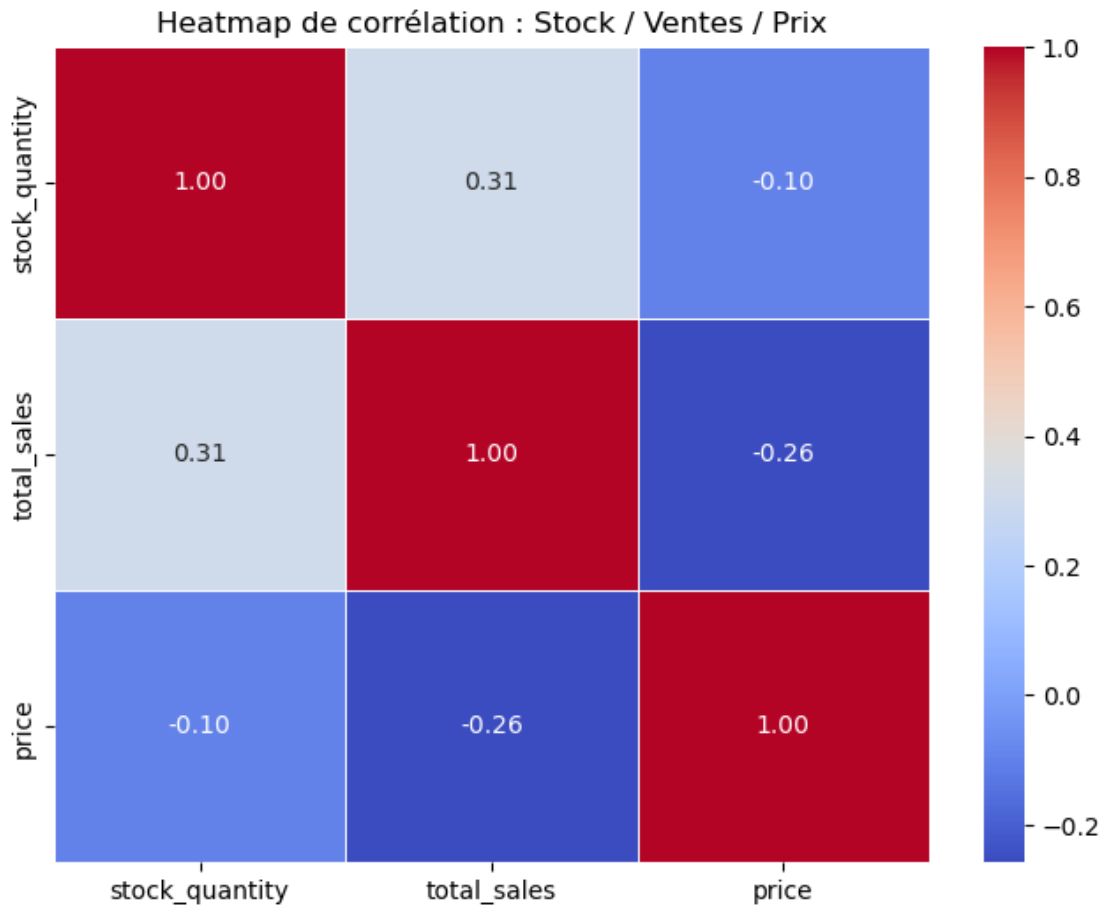
# Affichage de la heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    annot=True,          # affiche les valeurs dans les cases
    cmap="coolwarm",     # palette rouge → bleu
```

```

        linewidths=0.5,
        fmt=".2f"           # arrondi à 2 décimales
    )

plt.title("Heatmap de corrélation : Stock / Ventes / Prix")
plt.show()

```



```

[215]: # Sélection des variables pour la corrélation
cols = ["stock_quantity", "total_sales", "price", "purchase_price",
        ↪ "taux_marge"]

# Matrice de corrélation
corr_matrix = df_taux_pos[cols].corr()

# Affichage de la heatmap
plt.figure(figsize=(9, 7))
sns.heatmap(
    corr_matrix,

```

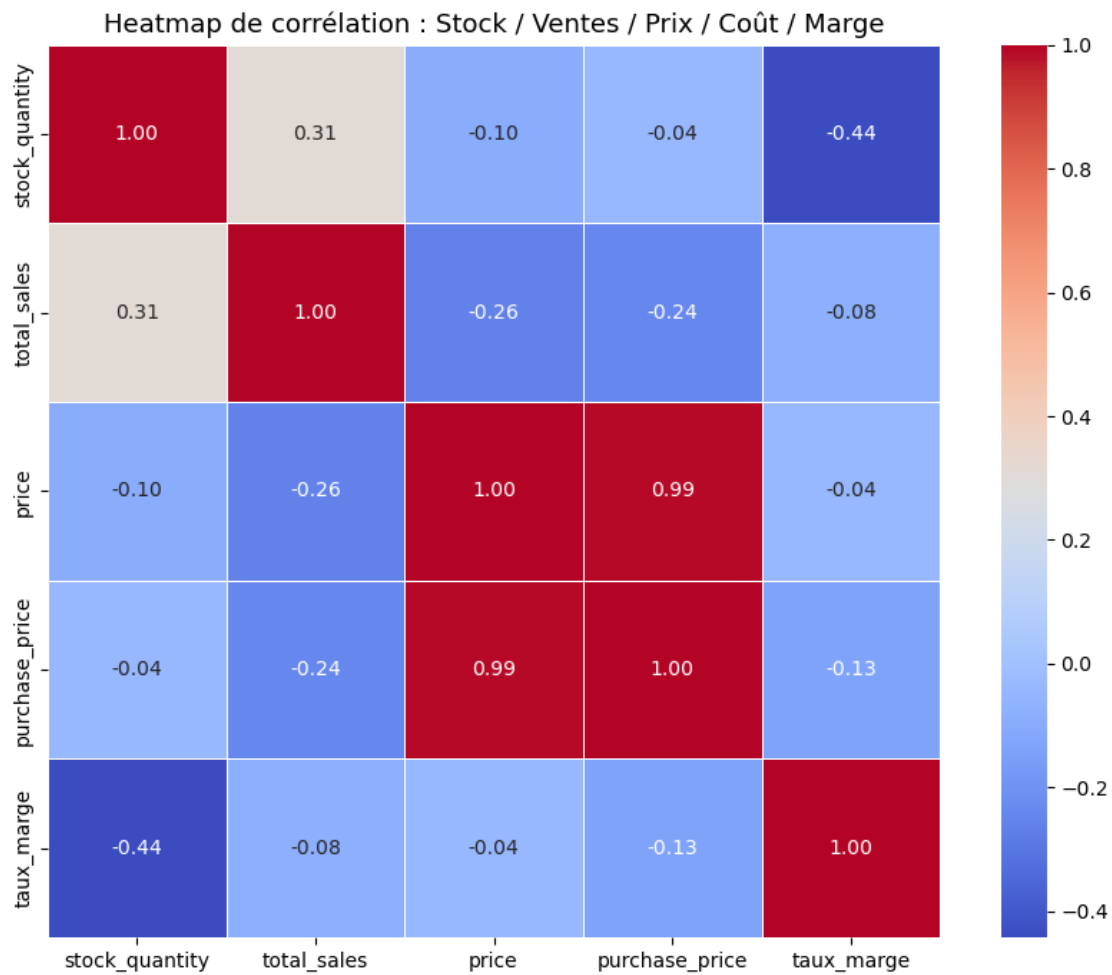


```

    annot=True,          # affiche les valeurs
    fmt=".2f",           # 2 décimales
    cmap="coolwarm",     # bleu rouge
    linewidths=0.5,
    square=True
)

plt.title("Heatmap de corrélation : Stock / Ventes / Prix / Coût / Marge",
          fontsize=13)
plt.tight_layout()
plt.show()

```



```
[216]: df_merge.to_excel("df_merge.xlsx", index=False)
```

```
[ ]:
```